The Concise TypeScript Book

Simone Poggiali

كتاب TypeScript الموجز

يقدّم كتاب TypeScript الموجز نظرة شاملة ومختصرة على إمكانات TypeScript. ويقدّم شروحات واضحة تغطي جميع جوانب أحدث إصدار من اللغة، بدءًا من نظام الأنواع القوي وصولًا إلى الميزات المتقدمة.

سواء أكنت مبتدئًا أم مطوّرًا متمرّسًا، فإن هذا الكتاب مورد قيّم للغاية لتعزيز فهمك وإتقانك للغة TypeScript.

هذا الكتاب مجاني بالكامل ومفتوح المصدر.

أؤمن بأن التعليم التقني عالي الجودة ينبغي أن يكون متاحًا للجميع. ولهذا السبب، أبقي الكتاب متاحًا مجانًا وأحدّثه بانتظام بإدخال تحسينات وأمثلة جديدة.

اكتشف **الإصدار Plus Edition من كتاب TypeScript الموجز**.



بالنسبة إلى القرّاء الذين يريدون تجاوز الإصدار المفتوح المصدر، يتضمن **الإصدار Plus Edition من كتاب TypeScript الموجز: React وأنماط واقعية لـ TypeScript 7** محتوى إضافيًا وحصريًا يركّز على التطبيق العملي.

يتضمن الإصدار Plus Edition ما يلي:

- **محدّث من أجل TypeScript 7** — تغطية لأحدث ميزات TypeScript 7 والتحسينات التي أُدخلت على اللغة.
- **TypeScript مع React** — إرشادات عملية لتعريف أنواع المكوّنات، وprops، وhooks، والأحداث، وchildren، وrefs، وأنماط React الشائعة.
- **وصفات TypeScript للمشروعات الواقعية** — أمثلة مركّزة تعالج المشكلات العملية التي يواجهها المطوّرون عند بناء تطبيقات

TypeScript وصيانتها.

من خلال شراء الإصدار Plus Edition، فإنك تدعم مباشرةً أيضًا استمرار تطوير الكتاب المجاني مفتوح المصدر وصيانته.

يتوفر الإصدار Plus Edition باللغتين الإنجليزية والإيطالية على Amazon في جميع أنحاء العالم. استكشف الإصدار Plus Edition واشتره من Amazon.

دعم المشروع

إذا ساعدك الكتاب المجاني على إصلاح خطأ برمجي، أو فهم مفهوم صعب، أو التقدّم في مسيرتك المهنية، فيُرجى التفكير في دعم عملي بدفع المبلغ الذي تريده، مع مساهمة مقترحة قدرها 5$، أو برعاية فنجان قهوة.

يساعدني دعمك على إبقاء المحتوى محدّثًا وتوسيعه بأمثلة جديدة، وشروحات أوضح، وإرشادات عملية إضافية.





الترجمات

تُرجم هذا الكتاب إلى عدة لغات، منها:

الإنجليزية

البلغارية

الألمانية

الفرنسية

الإندونيسية

الإيطالية

اليابانية

الكورية

البولندية

البرتغالية (البرازيل)

السويدية

التركية

الفيتنامية

الصينية

الإسبانية

التايلاندية

الروسية

العربية التشيكية

التنزيلات والموقع الإلكتروني

يمكنك أيضًا تنزيل إصدار EPUB:

https://github.com/gibbok/typescript-book/tree/main/downloads

يتوفر إصدار عبر الإنترنت على:

https://gibbok.github.io/typescript-book

جدول المحتويات

- كتاب TypeScript الموجز
  - دعم المشروع
  - الترجمات
  - التنزيلات والموقع الإلكتروني
  - جدول المحتويات
  - مقدمة
  - عن المؤلف
  - مقدمة إلى TypeScript
    - ما TypeScript؟
    - لماذا TypeScript؟
    - TypeScript وJavaScript
    - توليد الشيفرة في TypeScript
    - JavaScript الحديثة الآن (التحويل إلى إصدار أقدم)
  - بدء استخدام TypeScript
    - التثبيت
    - التهيئة
    - ملف تهيئة TypeScript
      - target
      - lib
      - strict
      - module
      - moduleResolution
      - esModuleInterop
      - jsx
      - skipLibCheck
      - files
      - include

        - exclude
    - importHelpers
    - نصائح للانتقال إلى TypeScript
- استكشاف نظام الأنواع
    - خدمة لغة TypeScript
    - التنميط البنيوي
    - قواعد المقارنة الأساسية في TypeScript
    - الأنواع بوصفها مجموعات
    - إسناد نوع: تصريحات الأنواع وتوكيدات الأنواع
        - تصريح النوع
        - توكيد النوع
        - التصريحات المحيطية
    - فحص الخصائص وفحص الخصائص الزائدة
    - الأنواع الضعيفة
    - الفحص الصارم للكائنات الحرفية (الحداثة)
    - استدلال الأنواع
    - استدلالات أكثر تقدمًا
    - توسيع النوع
    - Const
        - معدِّل Const على معاملات الأنواع
        - توكيد Const
    - التعليق التوضيحي الصريح للنوع
    - تضييق النوع
        - الشروط
        - طرح استثناء أو الإرجاع
        - الاتحاد المميَّز
        - حراس الأنواع المعرَّفة من قِبل المستخدم
        - التضييق باستخدام Switch-true
- الأنواع الأولية
    - string
    - boolean
    - number

- bigint
- Symbol
- null وundefined
- Array
- any
- التعليقات التوضيحية للأنواع
- الخصائص الاختيارية
- الخصائص المخصّصة للقراءة فقط
- تواقيع الفهرسة
- توسعة الأنواع
- الأنواع الحرفية
- استدلال الأنواع الحرفية
- strictNullChecks
- التعدادات
  - التعدادات العددية
  - التعدادات النصية
  - التعدادات الثابتة
  - الربط العكسي
  - التعدادات المحيطية
  - الأعضاء المحسوبة والثابتة
- التضييق
  - حراس الأنواع باستخدام typeof
  - التضييق بحسب الصدقية
  - التضييق بحسب المساواة
  - التضييق باستخدام المعامل In
  - التضييق باستخدام instanceof
- الإسنادات
- تحليل تدفق التحكم
- مسندات الأنواع
- الاتحادات المميَّزة
- النوع never
- التحقق من الشمول

- أنواع الكائنات
- نوع Tuple (مجهول الاسم)
- نوع Tuple المسمّى (الموسوم)
- Tuple ثابت الطول
- نوع الاتحاد
- أنواع التقاطع
- فهرسة الأنواع
- النوع من القيمة
- النوع من القيمة المرجعة للدالة
- النوع من الوحدة
- الأنواع المعيّنة
- معدِّلات الأنواع المعيّنة
- الأنواع الشرطية
- الأنواع الشرطية التوزيعية
- استدلال النوع باستخدام infer في الأنواع الشرطية
- الأنواع الشرطية المعرَّفة مسبقًا
- أنواع اتحادات القوالب
- النوع Any
- النوع Unknown
- النوع Void
- نوع Never
- الواجهة والنوع
  - الصياغة الشائعة
  - الأنواع الأساسية
  - الكائنات والواجهات
  - أنواع الاتحاد والتقاطع
- الأنواع الأولية المضمّنة
- كائنات JS المضمّنة الشائعة
- التحميلات الزائدة
- الدمج والتوسيع
- الاختلافات بين النوع والواجهة
- الصنف

- الصياغة الشائعة للصنف
- المُنشئ
- المُنشئات الخاصة والمحمية
- محددات الوصول
- الجلب والضبط
- الموصلات التلقائية في الأصناف
- this
- خصائص المعاملات
- الأصناف المجرّدة
- باستخدام الأنواع العامة
- المزخرفات
  - مزخرفات الأصناف
  - مزخرف الخاصية
  - مزخرف الأسلوب
  - مزخرفات دوال الجلب والضبط
  - البيانات الوصفية للمزخرفات
- الوراثة
- الأعضاء الساكنة
- تهيئة الخصائص
- التحميل الزائد للأساليب

- الأنواع العامة
  - النوع العام
  - الأصناف العامة
  - قيود الأنواع العامة
  - التضييق السياقي للأنواع العامة
- الأنواع البنيوية الممحوة
- مساحات الأسماء
- الرموز
- توجيهات الشرطات المائلة الثلاث
- معالجة الأنواع
  - إنشاء أنواع من أنواع أخرى
  - أنواع الوصول المفهرس

- أنواع الأدوات المساعدة
  - <Awaited<T
  - <Partial<T
  - <Required<T
  - <Readonly<T
  - <Record<K, T
  - <Pick<T, K
  - <Omit<T, K
  - <Exclude<T, U
  - <Extract<T, U
  - <NonNullable<T
  - <Parameters<T
  - <ConstructorParameters<T
  - <ReturnType<T
  - <InstanceType<T
  - <ThisParameterType<T
  - <OmitThisParameter<T
  - <ThisType<T
  - <Uppercase<T
  - <Lowercase<T
  - <Capitalize<T
  - <Uncapitalize<T
  - <NoInfer<T
- مواضيع أخرى
  - الأخطاء ومعالجة الاستثناءات
  - أصناف المزج
  - ميزات اللغة غير المتزامنة
  - المكرِّرات والمولِّدات
  - مرجع TsDocs وJSDoc
  - types@
  - JSX
  - وحدات ES6

- عامل الأَس في ES7
- عبارة for-await-of
- الخاصية الوصفية new.target
- تعبيرات الاستيراد الديناميكي
- "tsc –watch"
- عامل التوكيد بعدم انعدام القيمة
- التصريحات ذات القيم الافتراضية
- التسلسل الاختياري
- عامل الدمج المنعدم
- أنواع قوالب النصوص الحرفية
- التحميل الزائد للدوال
- الأنواع العودية
- الأنواع الشرطية العودية
- دعم وحدات ECMAScript في Node
- دوال التوكيد
- أنواع الصفوف متغيرة الطول
- الأنواع المُغلَّفة
- التغاير والتغاير العكسي في TypeScript
  - تعليقات التغاير الاختيارية لمعاملات الأنواع
- تواقيع الفهرس ذات أنماط السلاسل القالبية
- المعامل satisfies
- عمليات الاستيراد والتصدير الخاصة بالأنواع فقط
- تصريح using والإدارة الصريحة للموارد
  - تصريح await using
- سمات الاستيراد
- التحقق من صياغة التعبيرات النمطية
- import defer

مقدمة

مرحبًا بك في كتاب TypeScript الموجز! يزوّدك هذا الدليل بالمعرفة الأساسية والمهارات العملية اللازمة لتطوير تطبيقات TypeScript بفعالية. اكتشف المفاهيم والتقنيات الأساسية لكتابة شيفرة نظيفة ومتينة. سواء أكنت مبتدئًا أم مطوّرًا متمرّسًا، فإن هذا الكتاب يشكّل دليلًا شاملًا ومرجعًا مفيدًا في آن واحد للاستفادة من قوة TypeScript في مشروعاتك.

يغطي هذا الكتاب TypeScript 7.0.

عن المؤلف

Simone Poggiali هو مهندس برمجيات رئيسي متمرّس وشغوف بكتابة شيفرة برمجية احترافية منذ تسعينيات القرن الماضي. وقد ساهم طوال مسيرته المهنية الدولية في مشروعات عديدة لمجموعة واسعة من العملاء، بدءًا من الشركات الناشئة وصولًا إلى المؤسسات الكبرى. واستفادت شركات بارزة مثل HelloFresh وSiemens وO2 وLeroy Merlin وSnowplow من خبرته وتفانيه.

يمكنك التواصل مع Simone Poggiali عبر المنصات التالية:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- البريد الإلكتروني: gibbok.coding gmail.com

القائمة الكاملة للمساهمين: https://github.com/gibbok/typescript-book/graphs/contributors

مقدمة إلى TypeScript

ما TypeScript؟

TypeScript لغة برمجة ذات تنميط قوي مبنية على JavaScript. صمّمها Anders Hejlsberg في الأصل عام 2012، وتتولى Microsoft حاليًا تطويرها

وصيانتها بوصفها مشروعًا مفتوح المصدر.

تُصرَّف TypeScript إلى JavaScript ويمكن تنفيذها في أي بيئة تشغيل JavaScript (مثل المتصفح أو Node.js على خادم).

تدعم أنماط برمجة متعددة، مثل البرمجة الوظيفية، والعامة، والأمرية، وكائنية التوجّه، وهي لغة مصرّفة (محوّلة) تُحوّل إلى JavaScript قبل التنفيذ.

لماذا TypeScript؟

TypeScript لغة ذات تنميط قوي تساعد على منع أخطاء البرمجة الشائعة وتجنّب أنواع معيّنة من أخطاء وقت التشغيل قبل تنفيذ البرنامج.

تتيح اللغة ذات التنميط القوي للمطوّر تحديد قيود وسلوكيات مختلفة للبرنامج في تعريفات أنواع البيانات، ما يسهّل التحقق من صحة البرمجيات ومنع العيوب. وتبرز قيمة ذلك بوجه خاص في التطبيقات واسعة النطاق.

من فوائد TypeScript:

- التنميط الساكن، مع تنميط قوي اختياري
- استدلال النوع
- الوصول إلى ميزات ES6 وES7
- التوافق عبر المنصات والمتصفحات
- دعم الأدوات باستخدام IntelliSense

TypeScript وJavaScript

تُكتب TypeScript في ملفات `ts.` أو `tsx.`، بينما تُكتب ملفات JavaScript في ملفات `js.` أو `jsx.`.

يمكن للملفات ذات الامتداد `tsx.` أو `jsx.` أن تحتوي على امتداد لصياغة JavaScript يُسمّى JSX، والمستخدم في React لتطوير واجهات المستخدم.

تُعد TypeScript، من حيث البنية، مجموعة شاملة من JavaScript (ECMAScript 2015) مع إضافة الأنواع. كل شيفرة JavaScript هي شيفرة TypeScript صالحة، لكن العكس ليس صحيحًا دائمًا.

على سبيل المثال، لنتناول دالة في ملف JavaScript بالامتداد `.js`، مثل الآتي:

```
const sum = (a, b) => a + b;
```

يمكن تحويل الدالة واستخدامها في TypeScript بتغيير امتداد الملف إلى `.ts`. لكن إذا أُضيفت تعليقات توضيحية لأنواع TypeScript إلى الدالة نفسها، فلن يمكن تنفيذها في أي بيئة تشغيل JavaScript من دون تصريف. ستنتج شيفرة TypeScript التالية خطأً في البنية إذا لم تُصرَّف:

```
const sum = (a: number, b: number): number => a + b;
```

صُممت TypeScript لاكتشاف أخطاء وقت التشغيل المحتملة في وقت التصريف، وذلك عبر السماح للمطوّرين بالتعبير عن مقاصدهم باستخدام التعليقات التوضيحية للأنواع. وإضافةً إلى ذلك، تستطيع TypeScript اكتشاف بعض المشكلات حتى عند عدم تقديم تعليقات توضيحية صريحة للأنواع، بفضل استدلال النوع. على سبيل المثال، لا تحدد مقتطفة الشيفرة التالية أي أنواع TypeScript:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

في هذه الحالة، تكتشف TypeScript خطأً وتبلغ عنه:

```
Property 'y' does not exist on type '{ x: number; }'.
```

يتأثر نظام الأنواع في TypeScript إلى حد كبير بسلوك JavaScript في وقت التشغيل. فمثلًا، تجري نمذجة معامل الجمع (+)، الذي يمكنه في JavaScript إجراء ربط النصوص أو الجمع العددي، بالطريقة نفسها في TypeScript:

```
const result = '1' + 1; // النتيجة من النوع string
```

اتخذ الفريق المسؤول عن TypeScript قرارًا متعمّدًا باعتبار الاستخدامات غير المعتادة في JavaScript أخطاءً. على سبيل المثال، لنتناول شيفرة JavaScript الصالحة التالية:

```
const result = 1 + true; // في JavaScript، 2 تساوي النتيجة
```

لكن TypeScript تُصدر خطأً:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

يحدث هذا الخطأ لأن TypeScript تفرض توافق الأنواع بصرامة، وتتعرّف في هذه الحالة على عملية غير صالحة بين عدد وقيمة منطقية.

توليد الشيفرة في TypeScript

يتحمل مصرّف TypeScript مسؤوليتين رئيسيتين: التحقق من أخطاء الأنواع والتصريف إلى JavaScript. وهاتان العمليتان مستقلتان إحداهما عن الأخرى. لا تؤثر الأنواع في تنفيذ الشيفرة داخل بيئة تشغيل JavaScript، لأنها تُمحى بالكامل أثناء التصريف. ولا يزال بإمكان TypeScript إخراج شيفرة JavaScript حتى في وجود أخطاء في الأنواع. فيما يلي مثال على شيفرة TypeScript تحتوي على خطأ في النوع:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // لا يمكن إسناد وسيط من النوع 'string
```

ومع ذلك، لا يزال بإمكانها إنتاج مخرجات JavaScript قابلة للتنفيذ:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

لا يمكن التحقق من أنواع TypeScript في وقت التشغيل. على سبيل المثال:

```ts
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // يشير 'Dog' هنا كقيمة إلى نوع فقط، لكنه يُستخدم.
        // ...
    }
};
```

نظرًا إلى أن الأنواع تُمحى بعد التصريف، فلا توجد طريقة لتشغيل هذه الشيفرة في JavaScript. وللتعرّف على الأنواع في وقت التشغيل، نحتاج إلى استخدام آلية أخرى. توفّر TypeScript عدة خيارات، ومن الخيارات الشائعة «الاتحاد الموسوم». على سبيل المثال:

```ts
interface Dog {
    kind: 'dog'; // اتحاد موسوم
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // اتحاد موسوم
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};
```

```
const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

الخاصية "kind" هي قيمة يمكن استخدامها في وقت التشغيل للتمييز بين الكائنات في JavaScript.

ومن الممكن أيضًا أن يكون لقيمة في وقت التشغيل نوع مختلف عن النوع المصرّح به في تصريح النوع. يحدث ذلك مثلًا إذا أساء المطوّر تفسير نوع واجهة API وأضاف إليه تعليقًا توضيحيًا غير صحيح.

TypeScript مجموعة شاملة من JavaScript، لذا يمكن استخدام الكلمة المفتاحية "class" بوصفها نوعًا وقيمةً في وقت التشغيل.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;
```

```
const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

في JavaScript، يمتلك "class" خاصية "prototype"، ويمكن استخدام المعامل "instanceof" لاختبار ما إذا كانت خاصية prototype لمنشئ ما تظهر في أي موضع ضمن سلسلة prototype لكائن.

ليس لـ TypeScript أي تأثير في أداء وقت التشغيل، إذ ستُمحى جميع الأنواع. لكن TypeScript تضيف بعض الحمل الزائد إلى وقت البناء.

JavaScript الحديثة الآن (التحويل إلى إصدار أقدم)

يمكن لـ TypeScript تصريف الشيفرة إلى أي إصدار منشور من JavaScript منذ ECMAScript 3 (1999). وهذا يعني أن TypeScript تستطيع تحويل الشيفرة التي تستخدم أحدث ميزات JavaScript إلى إصدارات أقدم، وهي عملية تُعرف باسم Downleveling. ويتيح ذلك استخدام JavaScript الحديثة مع الحفاظ على أقصى قدر من التوافق مع بيئات التشغيل القديمة.

من المهم ملاحظة أنه أثناء التحويل إلى إصدار أقدم من JavaScript، قد تولّد TypeScript شيفرة يمكن أن تفرض حملًا إضافيًا على الأداء مقارنةً بالتنفيذات الأصلية.

فيما يلي بعض ميزات JavaScript الحديثة التي يمكن استخدامها في TypeScript:

- وحدات ECMAScript بدلًا من استدعاءات رد النداء “define” بأسلوب AMD أو عبارات “require” في CommonJS.
- الأصناف بدلًا من prototypes.
- التصريح عن المتغيّرات باستخدام “let” أو “const” بدلًا من “var”.
- حلقة “for-of” أو “.forEach” بدلًا من حلقة “for” التقليدية.
- الدوال السهمية بدلًا من تعبيرات الدوال.
- إسناد التفكيك.
- الأسماء المختصرة للخصائص/الأساليب وأسماء الخصائص المحسوبة.
- المعاملات الافتراضية للدوال.

من خلال الاستفادة من ميزات JavaScript الحديثة هذه، يمكن للمطوّرين كتابة شيفرة أكثر تعبيرًا وإيجازًا في TypeScript.

بدء استخدام TypeScript

التثبيت

يوفّر Visual Studio Code دعمًا ممتازًا للغة TypeScript، لكنه لا يتضمن مصرّف TypeScript. لتثبيت مصرّف TypeScript، يمكنك استخدام مدير حزم مثل npm أو yarn:

```
npm install typescript --save-dev
```

أو

```
yarn add typescript --dev
```

تأكد من إيداع ملف القفل الناتج لضمان استخدام كل عضو في الفريق لإصدار TypeScript نفسه.

لتشغيل مصرّف TypeScript، يمكنك استخدام الأوامر التالية:

```
npx tsc
```

أو

```
yarn tsc
```

يُوصى بتثبيت TypeScript على مستوى المشروع بدلًا من تثبيتها عموميًا، لأن ذلك يوفّر عملية بناء يمكن التنبؤ بها بدرجة أكبر. لكن للاستخدامات العابرة، يمكنك استخدام الأمر التالي:

```
npx tsc
```

أو تثبيتها عموميًا:

```
npm install -g typescript
```

إذا كنت تستخدم Microsoft Visual Studio، فيمكنك الحصول على TypeScript بوصفها حزمة في NuGet لمشروعات MSBuild. نفّذ الأمر التالي في وحدة تحكم مدير حزم NuGet:

```
Install-Package Microsoft.TypeScript.MSBuild
```

أثناء تثبيت TypeScript، يُثبَّت ملفان تنفيذيان: "tsc" بوصفه مصرّف TypeScript و"tsserver" بوصفه خادم TypeScript المستقل. يحتوي الخادم المستقل على المصرّف وخدمات اللغة التي يمكن للمحرّرات وبيئات التطوير المتكاملة استخدامها لتوفير الإكمال الذكي للشيفرة.

إضافةً إلى ذلك، تتوفر عدة محوّلات متوافقة مع TypeScript، مثل Babel (عبر مكوّن إضافي) أو swc. ويمكن استخدام هذه المحوّلات لتحويل شيفرة TypeScript إلى لغات أو إصدارات مستهدفة أخرى.

أُعيدت كتابة TypeScript 7.0 بلغة Go بوصفها تنفيذًا أصليًا للمصرّف وخدمة اللغة. وهي تستخدم تعدد خيوط التنفيذ ذات الذاكرة المشتركة وتحسينات أخرى لجعل عمليات البناء الكاملة وميزات المحرّر أسرع، ما يقلل زمن الحصول على الملاحظات أثناء التطوير.

يمكن ضبط بعض ميزات الأداء في TypeScript 7.0. يمكن تشغيل التحقق من الأنواع في عمليات عاملة متوازية باستخدام `--checkers`؛ وقد يؤدي المزيد من العمليات العاملة إلى تسريع المشروعات الكبيرة، لكنه يستهلك ذاكرة أكبر. يحسّن وضع `--watch` المعاد بناؤه مراقبة الملفات عبر المنصات. لا تتضمن TypeScript 7.0 حتى الآن واجهة API للمصرّف (حتى يوليو 2026)، ولذلك يمكن للأدوات التي لا تزال تحتاج إلى واجهة API في TypeScript 6.0 تشغيلها بالتوازي مع TypeScript 7.0 باستخدام `@typescript/typescript6` أو الأسماء البديلة في npm.

التهيئة

يمكن تهيئة TypeScript باستخدام خيارات واجهة سطر أوامر tsc أو عبر استخدام ملف تهيئة مخصّص يُسمّى tsconfig.json ويوضع في جذر المشروع.

لإنشاء ملف tsconfig.json مجهّز مسبقًا بالإعدادات الموصى بها، يمكنك استخدام الأمر التالي:

```
tsc --init
```

عند تنفيذ الأمر `tsc` محليًا، ستصرّف TypeScript الشيفرة باستخدام التهيئة المحددة في أقرب ملف tsconfig.json.

فيما يلي بعض أمثلة أوامر واجهة سطر الأوامر التي تعمل بالإعدادات الافتراضية:

```
tsc main.ts // ترجمة ملف محدد (main.ts) إلى JavaScript
tsc src/*.ts // ترجمة ملفات .ts الموجودة في المجلد 'src' إلى
JavaScript
tsc app.ts util.ts --outfile index.js // ترجمة ملفي TypeScript
(app.ts و util.ts) إلى ملف JavaScript واحد (index.js)
```

ملف تهيئة TypeScript

يُستخدم ملف tsconfig.json لتهيئة مصرّف TypeScript (tsc). وعادةً ما يُضاف إلى جذر المشروع إلى جانب ملف package.json.

ملاحظات:

- يقبل tsconfig.json التعليقات حتى وإن كان بتنسيق json.
- يُنصح باستخدام ملف التهيئة هذا بدلًا من خيارات سطر الأوامر.

يمكنك العثور في الرابط التالي على الوثائق الكاملة والمخطط الخاص به:

https://www.typescriptlang.org/tsconfig

/https://www.typescriptlang.org/tsconfig

تمثّل القائمة التالية مجموعة من إعدادات التهيئة الشائعة والمفيدة:

target

تُستخدم الخاصية "target" لتحديد إصدار ECMAScript الذي ينبغي أن تُخرج أو تُصرّف إليه شيفرة TypeScript. وتُعد ES6 خيارًا جيدًا للمتصفحات الحديثة. ملاحظة: أُهمل دعم ES5 في TypeScript 6.0، ولم يعد مدعومًا في TypeScript 7.0.

lib

تُستخدم الخاصية "lib" لتحديد ملفات المكتبة التي يجب تضمينها في وقت التصريف. تضمّن TypeScript تلقائيًا واجهات API للميزات المحددة في الخاصية "target"، لكن يمكن استبعاد مكتبات بعينها أو اختيارها لتلبية احتياجات محددة. فإذا كنت تعمل على مشروع خادم مثلًا، يمكنك استبعاد مكتبة "DOM" التي لا تفيد إلا في بيئة المتصفح.

strict

يحسّن الخيار "strict" أمان الأنواع بتمكين عمليات تحقق أقوى. وهو مفعّل افتراضيًا بدءًا من TypeScript 6.0؛ وفي غير ذلك، ينبغي ضبطه صراحةً على true في ملف tsconfig.json. يتيح تمكين "strict" لـ TypeScript ما يلي:

- إخراج الشيفرة باستخدام "use strict" لكل ملف مصدر.
- مراعاة "null" و"undefined" في عملية التحقق من الأنواع.
- تعطيل استخدام النوع "any" عند عدم وجود تعليقات توضيحية للأنواع.
- إصدار خطأ عند استخدام التعبير "this" الذي كان سيعني خلاف ذلك النوع "any".

module

تحدد الخاصية "module" نظام الوحدات المدعوم للبرنامج المصرّف. وفي وقت التشغيل، يُستخدم محمّل وحدات لتحديد التبعيات وتنفيذها استنادًا إلى نظام الوحدات المحدد.

أكثر محمّلات الوحدات استخدامًا في JavaScript هما CommonJS في Node.js لتطبيقات جانب الخادم وRequireJS لوحدات AMD في تطبيقات الويب القائمة على المتصفح. ويمكن لـ TypeScript إخراج شيفرة لأنظمة وحدات مختلفة، منها UMD وSystem وESNext وES2015/ES6 وES2020. وينبغي اختيار نظام الوحدات استنادًا إلى البيئة المستهدفة وآلية تحميل الوحدات المتاحة فيها.

ملاحظة: أُهمل دعم أنظمة الوحدات القديمة (AMD وUMD وSystemJS) في TypeScript 6.0، ولم تعد مدعومة في TypeScript 7.0.

moduleResolution

تحدد الخاصية "moduleResolution" استراتيجية تحليل الوحدات. استخدم "nodenext" أو "bundler" لشيفرة TypeScript الحديثة. ولا تُستخدم الاستراتيجية "classic" إلا مع الإصدارات القديمة من TypeScript (قبل 1.6).

esModuleInterop

تسمح الخاصية "esModuleInterop" بعمليات الاستيراد الافتراضية من وحدات CommonJS التي لم تُصدّر باستخدام الخاصية "default"؛ وتوفّر هذه الخاصية طبقة توافق لضمان التوافق في JavaScript الناتجة. وبعد تمكين هذا الخيار، يمكننا استخدام `import MyLibrary from "my-library"` بدلًا من `import * as MyLibrary from "my-library"`.

كانت "esModuleInterop" في الأصل خيارًا صريحًا لتجنّب التغييرات الكاسرة، لكنها أصبحت منذ وقت طويل الإعداد الافتراضي الموصى به. وقد يؤدي تعطيلها إلى مشكلات دقيقة في وقت التشغيل عند استخدام CommonJS مع ESM. ملاحظة: بدءًا من TypeScript 6.0، يكون سلوك التشغيل البيني الأكثر أمانًا هذا مفعّلًا دائمًا.

في TypeScript 6.0، أُهملت بعض خيارات التهيئة وأشكال البنية القديمة أو مرّت بمرحلة انتقالية عبر السلوك القديم. وفي TypeScript 7.0، أصبحت أخطاءً مانعة أو ذات سلوك لا يُجري أي عملية.

الخيارات المهملة التي تحوّلت إلى أخطاء مانعة ذات سلوك لا يُجري أي عملية هي:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- تعطيل `esModuleInterop` أو `allowSyntheticDefaultImports`
- تعطيل `alwaysStrict`
- الكلمة المفتاحية `module` في تصريحات مساحة الاسم
- `asserts` في عمليات الاستيراد
- `/// <reference no-default-lib />` ضمن `skipDefaultLibCheck`

- مسارات ملفات واجهة سطر الأوامر التي تحتوي محليًا على `tsconfig.json` ما لم يُستخدم `--ignoreConfig`

jsx

تنطبق الخاصية "jsx" فقط على ملفات .tsx المستخدمة في ReactJS، وتتحكم في كيفية تصريف بُنى JSX إلى JavaScript. ومن الخيارات الشائعة "preserve" الذي يصرّف إلى ملف .jsx مع إبقاء JSX من دون تغيير، بحيث يمكن تمريره إلى أدوات مختلفة مثل Babel لإجراء المزيد من التحويلات.

skipLibCheck

تمنع الخاصية "skipLibCheck" TypeScript من التحقق من أنواع حزم الجهات الخارجية المستوردة بأكملها. وتقلل هذه الخاصية وقت تصريف المشروع. وستظل TypeScript تتحقق من شيفرتك في مقابل تعريفات الأنواع التي توفّرها هذه الحزم.

files

تشير الخاصية "files" للمصرّف إلى قائمة الملفات التي يجب تضمينها دائمًا في البرنامج.

include

تشير الخاصية "include" للمصرّف إلى قائمة الملفات التي نرغب في تضمينها. وتسمح هذه الخاصية بأنماط شبيهة بأنماط glob، مثل "*" *لأي دليل فرعي*، و"" لأي اسم ملف، و"?" للمحارف الاختيارية.

exclude

تشير الخاصية "exclude" للمصرّف إلى قائمة الملفات التي ينبغي عدم تضمينها في التصريف. ويمكن أن تشمل ملفات مثل "node_modules" أو

ملفات الاختبار. ملاحظة: يسمح tsconfig.json بالتعليقات.

importHelpers

تستخدم TypeScript شيفرة مساعدة عند توليد الشيفرة لبعض ميزات JavaScript المتقدمة أو الميزات التي تُحوَّل إلى إصدارات أقدم. افتراضيًا، تتكرر هذه الدوال المساعدة في الملفات التي تستخدمها. أما الخيار `importHelpers` فيستورد هذه الدوال المساعدة من الوحدة `tslib` بدلًا من ذلك، ما يجعل مخرجات JavaScript أكثر كفاءة.

نصائح للانتقال إلى TypeScript

بالنسبة إلى المشاريع الكبيرة، يُنصح باتباع انتقال تدريجي تتعايش فيه شيفرات TypeScript وJavaScript في البداية. ولا يمكن الانتقال إلى TypeScript دفعة واحدة إلا في المشاريع الصغيرة.

تتمثل الخطوة الأولى في هذا الانتقال في إدخال TypeScript ضمن سلسلة عملية البناء. ويمكن فعل ذلك باستخدام خيار المصرّف "allowJs"، الذي يسمح لملفات .ts و.tsx بالتعايش مع ملفات JavaScript الموجودة. وبما أن TypeScript سيلجأ إلى النوع "any" للمتغير عندما يتعذر عليه استدلال النوع من ملفات JavaScript، فيُنصح بتعطيل "noImplicitAny" في خيارات المصرّف في بداية الانتقال.

تتمثل الخطوة الثانية في التأكد من أن اختبارات JavaScript تعمل إلى جانب ملفات TypeScript، حتى تتمكن من تشغيل الاختبارات أثناء تحويل كل وحدة. وإذا كنت تستخدم Jest، ففكّر في استخدام `ts-jest`، الذي يتيح لك اختبار مشاريع TypeScript باستخدام Jest.

تتمثل الخطوة الثالثة في تضمين تصريحات الأنواع الخاصة بمكتبات الجهات الخارجية في مشروعك. ويمكن العثور على هذه التصريحات إما مضمّنة مع المكتبات أو على DefinitelyTyped. ويمكنك البحث عنها باستخدام وتثبيتها باستخدام: https://www.typescriptlang.org/dt/search

```
npm install --save-dev @types/package-name
```

أو

```
yarn add --dev @types/package-name
```

تتمثل الخطوة الرابعة في الانتقال وحدةً تلو الأخرى باتباع نهج من الأسفل إلى الأعلى، مع تتبّع مخطط التبعيات بدءًا من الأوراق. والفكرة هي البدء بتحويل الوحدات التي لا تعتمد على وحدات أخرى. ولعرض مخططات التبعيات مرئيًا، يمكنك استخدام أداة "madge".

من الوحدات المرشحة جيدًا لهذه التحويلات الأولية دوال الأدوات المساعدة والشيفرة المرتبطة بواجهات API الخارجية أو المواصفات. ومن الممكن توليد تعريفات أنواع TypeScript تلقائيًا من عقود Swagger أو مخططات GraphQL أو JSON لتضمينها في مشروعك.

عندما لا تتوفر مواصفات أو مخططات رسمية، يمكنك توليد الأنواع من البيانات الأولية، مثل JSON الذي يعيده الخادم. ومع ذلك، يُنصح بتوليد الأنواع من المواصفات بدلًا من البيانات لتجنب إغفال الحالات الحدّية.

أثناء الانتقال، امتنع عن إعادة هيكلة الشيفرة وركّز فقط على إضافة الأنواع إلى وحداتك.

تتمثل الخطوة الخامسة في تفعيل "noImplicitAny"، ما سيفرض معرفة جميع الأنواع وتعريفها، ويوفر تجربة أفضل لاستخدام TypeScript في مشروعك.

أثناء الانتقال، يمكنك استخدام التوجيه `@ts-check`، الذي يفعّل التحقق من أنواع TypeScript داخل ملف JavaScript. يوفر هذا التوجيه إصدارًا متساهلًا من التحقق من الأنواع، ويمكن استخدامه في البداية لتحديد المشكلات في ملفات JavaScript. وعندما يُضمَّن `@ts-check` في ملف، ستحاول TypeScript استدلال التعريفات باستخدام تعليقات بأسلوب JSDoc. ومع ذلك، فكّر في استخدام تعليقات JSDoc التوضيحية في مرحلة مبكرة جدًا من الانتقال فقط.

فكّر في إبقاء القيمة الافتراضية للخيار `noEmitOnError` في ملف tsconfig.json على false. سيسمح لك ذلك بإخراج شيفرة JavaScript المصدرية حتى عند الإبلاغ عن أخطاء.

استكشاف نظام الأنواع

خدمة لغة TypeScript

تقدم خدمة لغة TypeScript، المعروفة أيضًا باسم tsserver، ميزات متعددة مثل الإبلاغ عن الأخطاء، والتشخيصات، والتصريف عند الحفظ، وإعادة التسمية، والانتقال إلى التعريف، وقوائم الإكمال، والمساعدة الخاصة بالتواقيع، وغيرها. وتستخدمها أساسًا بيئات التطوير المتكاملة (IDEs) لتوفير دعم IntelliSense. وهي تتكامل بسلاسة مع Visual Studio Code وتستخدمها أدوات مثل Coc (Conquer of Completion).

يمكن للمطوّرين الاستفادة من واجهة API مخصصة وإنشاء إضافاتهم الخاصة والمخصصة لخدمة اللغة لتحسين تجربة تحرير TypeScript. وقد يكون ذلك مفيدًا على نحو خاص لتنفيذ ميزات تدقيق خاصة أو لتمكين الإكمال التلقائي للغة قوالب مخصصة.

من الأمثلة الواقعية على الإضافات المخصصة "typescript-styled-plugin"، التي توفر الإبلاغ عن أخطاء بناء الجملة ودعم IntelliSense لخصائص CSS في المكونات المنسّقة.

لمزيد من المعلومات وأدلة البدء السريع، يمكنك الرجوع إلى ويكي TypeScript الرسمية على GitHub:
https://github.com/microsoft/TypeScript/wiki/

التنميط البنيوي

تعتمد TypeScript على نظام أنواع بنيوي. وهذا يعني أن توافق الأنواع وتكافؤها يتحددان من خلال البنية أو التعريف الفعلي للنوع، وليس اسمه أو موضع إعلانه، كما في أنظمة الأنواع الاسمية مثل C# أو C.

صُمم نظام الأنواع البنيوي في TypeScript استنادًا إلى طريقة عمل نظام التنميط الديناميكي القائم على التوافق البنيوي في JavaScript أثناء وقت التشغيل.

المثال التالي هو شيفرة TypeScript صالحة. وكما تلاحظ، يحتوي "X" و"Y" على العضو نفسه "a"، رغم اختلاف اسمي التصريح عنهما. تتحدد الأنواع من خلال بنيتها، وفي هذه الحالة، بما أن البنيتين متماثلتان، فهما متوافقتان وصالحتان.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // صحيح
```

قواعد المقارنة الأساسية في TypeScript

عملية المقارنة في TypeScript تعاودية وتُنفذ على الأنواع المتداخلة في أي مستوى.

يكون النوع "X" متوافقًا مع "Y" إذا كان "Y" يحتوي على الأقل على أعضاء "X" نفسها.

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // ي الأقل على الأعضاء نفسها الموجودة في
const r: X = y;
```

تُقارن معاملات الدوال حسب أنواعها، وليس حسب أسمائها:

```ts
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // صحيح
x = y; // صحيح
```

يجب أن تكون أنواع إرجاع الدوال متماثلة:

```ts
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // غير صحيح
x = y; // غير صحيح
```

يجب أن يكون نوع إرجاع الدالة المصدر نوعًا فرعيًا من نوع إرجاع الدالة الهدف:

```ts
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // صحيح
y = x; // مفقود b غير صحيح، العضو
```

يُسمح بتجاهل معاملات الدوال، إذ إنها ممارسة شائعة في JavaScript، مثل استخدام "Array.prototype.map()":

```ts
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

وبالتالي، تكون تصريحات الأنواع التالية صالحة تمامًا:

```ts
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // مفقود b المعامل
y = x; // صحيح
```

تكون أي معاملات اختيارية إضافية في النوع المصدر صالحة:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // صحيح
x = y; //صحيح
```

تُعد أي معاملات اختيارية في النوع الهدف لا تقابلها معاملات في النوع المصدر صالحة، ولا تمثل خطأ:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // صحيح
x = y; // صحيح
```

يُعامل معامل البقية كسلسلة لا نهائية من المعاملات الاختيارية:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //صحيح
```

تكون الدوال ذات الأحمال الزائدة صالحة إذا كان توقيع الحمل الزائد متوافقًا مع توقيع تنفيذها:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // صحيح
x('a', 1); // صحيح

function y(a: string): void; // غير صحيح، غير متوافق مع توقيع التنفيذ
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
```

```
y('a');
y('a', 1);
```

تنجح مقارنة معاملات الدوال إذا كان من الممكن إسناد المعاملات المصدر والهدف إلى الأنواع الفائقة أو الأنواع الفرعية (التباين الثنائي).

```
// نوع أعلى
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// نوع فرعي
class Y extends X {}
// نوع فرعي
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// التغاير ثنائي الاتجاه يقبل الأنواع الأعلى
console.log(getA(new X('x'))); // صحيح
console.log(getA(new Y('Y'))); // صحيح
console.log(getA(new Z('z'))); // صحيح
```

يمكن مقارنة التعدادات بالأرقام والعكس صحيح ويكون ذلك صالحًا، لكن مقارنة قيم تعداد من أنواع تعداد مختلفة غير صالحة.

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
```

```
const xa: number = X.A; // صحيح
const ya: Y = 0; // صحيح
X.A === Y.A; // غير صحيح
```

تخضع مثيلات الصنف لفحص توافق أعضائها الخاصة والمحمية:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // غير صحيح
```

لا يأخذ فحص المقارنة في الحسبان اختلاف التسلسل الهرمي للوراثة، على سبيل المثال:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
```

```
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // صحيح
x === z; // صحيح حتى لو كان z مختلف من تسلسل وراثة
```

تُقارن الأنواع العامة باستخدام بنيتها استنادًا إلى النوع الناتج بعد تطبيق معامل النوع العام، ولا يُقارن إلا الناتج النهائي بوصفه نوعًا غير عام.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // غير صحيح لأن وسيط النوع يُستخدم في البنية النهائية

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // صحيح لأن وسيط النوع لا يُستخدم في البنية النهائية
```

عندما لا يُحدَّد وسيط النوع للأنواع العامة، تُعامل جميع الوسائط غير المحددة على أنها من النوع "any":

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // صحيح
```

تذكّر:

```
let a: number = 1:
```

```
let b: number = 2;
a = b; // صحيح، يمكن إسناد كل شيء إلى نفسه

let c: any;
c = 1; // صحيح، يمكن إسناد جميع الأنواع إلى any

let d: unknown;
d = 1; // صحيح، يمكن إسناد جميع الأنواع إلى unknown

let e: unknown;
let e1: unknown = e; // صحيح، لا يمكن إسناد unknown وإلى نفسه إلا إلى
let e2: any = e; // صحيح
let e3: number = e; // غير صحيح

let f: never;
f = 1; // غير صحيح، لا يمكن إسناد أي شيء إلى never

let g: void;
let g1: any;
g = 1; // غير صحيح، لا يمكن الإسناد من void باستثناء any أو إليه
g = g1; // صحيح
```

يرجى ملاحظة أنه عند تفعيل "strictNullChecks"، يُعامل "null" و"undefined" على نحو مماثل لـ "void"؛ وإلا فيُعاملان على نحو مماثل لـ "never".

الأنواع بوصفها مجموعات

في TypeScript، النوع هو مجموعة من القيم الممكنة. ويُشار إلى هذه المجموعة أيضًا باسم مجال النوع. ويمكن النظر إلى كل قيمة من نوع ما على أنها عنصر في مجموعة. ويحدد النوع القيود التي يجب أن يستوفيها كل عنصر في المجموعة حتى يُعد عضوًا فيها. تتمثل المهمة الأساسية لـ TypeScript في فحص ما إذا كانت إحدى المجموعات مجموعة جزئية من أخرى والتحقق من ذلك.

تدعم TypeScript أنواعًا مختلفة من المجموعات:

مصطلح المجموعة	TypeScript	ملاحظات
المجموعة الخالية	never	لا يحتوي "never" على أي شيء باستثناء نفسه
مجموعة أحادية العنصر	undefined / null / literal type	
مجموعة منتهية	boolean / union	
مجموعة غير منتهية	string / number / object	
المجموعة الشاملة	any / unknown	كل عنصر عضو في "any" وكل مجموعة مجموعة جزئية منه / يُعد "unknown" النظير الآمن من حيث الأنواع لـ "any"

فيما يلي بعض الأمثلة:

TypeScript	مصطلح المجموعة	مثال
never	∅ (المجموعة الخالية)	const x: never = 'x'; // Error: Type 'string' is not assignable to type 'never'
Literal type	مجموعة أحادية العنصر	type X = 'X'; type Y = 7;
Value assignable to T	القيمة T ∋ (عضو في)	type XY = 'X' | 'Y'; const x: XY = 'X';
T1 assignable to T2	T1 ⊆ T2 (مجموعة جزئية من)	type XY = 'X' | 'Y'; const x: XY = 'X'; const j: XY = 'J'; // Type "'J"' is not assignable to type 'XY'.

T1 extends T2	T1 ⊆ T2 (مجموعة جزئية من)	;type X = ‘X’ extends string ? true : false
T1 | T2	T1 ∪ T2 (اتحاد)	;’type XY = ‘X’ | ‘Y ;type JK = 1 | 2
T1 & T2	T1 ∩ T2 (تقاطع)	{ type X = { a: string { type Y = { b: string type XY = X & Y { ’const x: XY = { a: ‘a’, b: ‘b
unknown	المجموعة الشاملة	const x: unknown = 1

ينشئ الاتحاد، (T1 | T2)، مجموعة أوسع (كلاهما):

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // صحيح
```

ينشئ التقاطع، (T1 & T2)، مجموعة أضيق (المشترك فقط):

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // غير صحيح
const j: XY = { a: 'a', b: 'b' }; // صحيح
```

يمكن اعتبار الكلمة المفتاحية extends بمعنى “مجموعة جزئية من” في هذا السياق. فهي تضع قيدًا لنوع ما. وعندما تُستخدم extends مع نوع عام، فإنها

تقيّد معامل النوع العام بنوع أكثر تحديدًا.

يرجى ملاحظة أن `extends` هنا لا علاقة لها بوراثة الأصناف بمعنى البرمجة كائنية التوجه.

تتعامل TypeScript مع الأنواع البنيوية، وليس لديها تسلسل هرمي اسمي صارم. وفي الواقع، كما في المثال أدناه، يمكن أن يتداخل نوعان من دون أن يكون أي منهما نوعًا فرعيًا للآخر، لأن TypeScript تأخذ بنية الكائنات أو شكلها في الحسبان.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // صحيح
```

إسناد نوع: تصريحات الأنواع وتوكيدات الأنواع

يمكن إسناد نوع بطرق مختلفة في TypeScript:

تصريح النوع

في المثال التالي، نستخدم `x: X` (أي: Type) للتصريح بنوع المتغير x.

```
type X = {
    a: string;
};

// تصريح نوع
const x: X = {
    a: 'a',
};
```

إذا لم يكن المتغير بالتنسيق المحدد، فستبلغ TypeScript عن خطأ. على سبيل المثال:

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
    b: 'b', // خطأ: لا يمكن للكائن الحرفي تحديد سوى الخصائص المعروفة
};
```

توكيد النوع

يمكن إضافة توكيد باستخدام الكلمة المفتاحية as. يخبر هذا المصرّف بأن لدى المطوّر معلومات أكثر عن النوع، ويمنع ظهور أي أخطاء قد تحدث.

على سبيل المثال:

```
type X = {
    a: string;
```

```
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

في المثال أعلاه، يُؤكَّد أن الكائن x من النوع X باستخدام الكلمة المفتاحية as. ويُعلم ذلك مصرّف TypeScript بأن الكائن يتوافق مع النوع المحدد، رغم احتوائه على الخاصية الإضافية b غير الموجودة في تعريف النوع.

تكون توكيدات الأنواع مفيدة في الحالات التي يلزم فيها تحديد نوع أكثر تخصيصًا، ولا سيما عند العمل مع DOM. على سبيل المثال:

```
const myInput = document.getElementById('my_input') as HTMLInput
```

هنا، يُستخدم توكيد النوع as HTMLInputElement لإخبار TypeScript بأن نتيجة getElementById ينبغي أن تُعامل على أنها HTMLInputElement. يمكن أيضًا استخدام توكيدات الأنواع لإعادة تعيين المفاتيح، كما هو موضح في المثال أدناه باستخدام القوالب النصية:

```
type J<Type> = {
    [
        Property in keyof Type as `prefix_${string & Property}`
    ]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

في هذا المثال، يستخدم النوع J<Type> نوعًا معيّنًا مع قالب نصي لإعادة تعيين مفاتيح Type. وينشئ خصائص جديدة يُضاف إليها "prefix_" في بداية كل مفتاح منها، وتكون قيمها المقابلة دوال تُرجع قيم الخصائص الأصلية.

تجدر الإشارة إلى أن TypeScript لن تنفذ فحص الخصائص الزائدة عند استخدام توكيد النوع. لذلك، يُفضّل عمومًا استخدام تصريح النوع عندما تكون بنية الكائن معروفة مسبقًا.

التصريحات المحيطية

التصريحات المحيطية هي ملفات تصف أنواع شيفرة JavaScript، ويكون تنسيق اسم الملف الخاص بها `..d.ts` وعادةً ما تُستورد وتُستخدم لإضافة تعليقات توضيحية إلى مكتبات JavaScript الموجودة أو لإضافة أنواع إلى ملفات JS الموجودة في مشروعك.

يمكن العثور على أنواع العديد من المكتبات الشائعة في:
/https://github.com/DefinitelyTyped/DefinitelyTyped

ويمكن تثبيتها باستخدام:

```
npm install --save-dev @types/library-name
```

بالنسبة إلى التصريحات المحيطية التي عرّفتها، يمكنك الاستيراد باستخدام مرجع "الشرطات الثلاث":

```
/// <reference path="./library-types.d.ts" />
```

يمكنك استخدام التصريحات المحيطية حتى داخل ملفات JavaScript باستخدام `// @ts-check`.

تتيح الكلمة المفتاحية `declare` تعريف الأنواع لشيفرة JavaScript موجودة من دون استيرادها، وتعمل بوصفها عنصرًا نائبًا للأنواع الواردة من ملف آخر أو المتاحة عموميًا.

فحص الخصائص وفحص الخصائص الزائدة

تعتمد TypeScript على نظام أنواع بنيوي، لكن فحص الخصائص الزائدة هو إحدى ميزاتها التي تتيح لها التحقق مما إذا كان الكائن يحتوي على الخصائص

المحددة في النوع بدقة.

يُجرى فحص الخصائص الزائدة عند إسناد الكائنات الحرفية إلى متغيرات، أو عند تمريرها بوصفها وسائط إلى دالة، مثلًا.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
const x: X = y; // صحيح بفضل نظام الأنواع البنيوي
const w: X = { a: 'a', b: 'b' }; // صحيح بسبب فحص الخصائص الزائدة
```

الأنواع الضعيفة

يُعد النوع ضعيفًا عندما لا يحتوي إلا على مجموعة من الخصائص الاختيارية بالكامل:

```
type X = {
    a?: string;
    b?: string;
};
```

تعد TypeScript إسناد أي شيء إلى نوع ضعيف خطأً عندما لا يوجد أي تداخل؛ فعلى سبيل المثال، يؤدي ما يلي إلى ظهور خطأ:

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // غير صحيح
```

رغم أن ذلك غير موصى به، يمكن عند الحاجة تجاوز هذا الفحص باستخدام توكيد النوع:

```ts
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // صحيح
```

أو بإضافة unknown إلى توقيع الفهرس في النوع الضعيف:

```ts
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // صحيح
```

الفحص الصارم للكائنات الحرفية (الحداثة)

الفحص الصارم للكائنات الحرفية، الذي يُشار إليه أحيانًا باسم "الحداثة"، هو ميزة في TypeScript تساعد على اكتشاف الخصائص الزائدة أو المكتوبة إملائيًا بصورة خاطئة، التي قد تمر دون ملاحظة في فحوص الأنواع البنيوية المعتادة.

عند إنشاء كائن حرفي، يعدّه مصرّف TypeScript "حديثًا". وإذا أُسند الكائن الحرفي إلى متغير أو مُرِّر بوصفه معاملًا، فستطرح TypeScript خطأً إذا حدد الكائن الحرفي خصائص غير موجودة في النوع الهدف.

لكن "الحداثة" تختفي عند توسيع كائن حرفي أو استخدام توكيد النوع.

فيما يلي بعض الأمثلة للتوضيح:

```ts
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
```

```
x = { a: 'a', b: 'b' }; // فحص الحداثة: إسناد غير صحيح
var y: Y;
y = { a: 'a', bx: 'bx' }; // فحص الحداثة: إسناد غير صحيح

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // توسيع النوع: لا أخطاء، الأنواع متوافقة بنيويًا

fn({ a: 'a', bx: 'b' }); // فحص الحداثة: وسيط غير صحيح

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // توسيع النوع: لا يوجد فحص للحداثة
```

استدلال الأنواع

يمكن لـ TypeScript استدلال الأنواع عند عدم توفير تعليق توضيحي في أثناء:

- تهيئة المتغير.
- تهيئة العضو.
- تعيين القيم الافتراضية للمعاملات.
- نوع إرجاع الدالة.

على سبيل المثال:

```
let x = 'x'; // النوع المستنتج هو string
```

يحلل مصرّف TypeScript القيمة أو التعبير، ويحدد نوعه استنادًا إلى المعلومات المتاحة.

استدلالات أكثر تقدمًا

عند استخدام عدة تعبيرات في استدلال الأنواع، تبحث TypeScript عن "أفضل الأنواع المشتركة". على سبيل المثال:

```
let x = [1, 'x', 1, null]; // النوع المستنتج هو: (string | number |
```

إذا لم يتمكن المصرّف من العثور على أفضل الأنواع المشتركة، فإنه يعيد نوع اتحاد. على سبيل المثال:

```
let x = [new RegExp('x'), new Date()]; // النوع المستنتج هو: (RegExp
```

تستخدم TypeScript "التنميط السياقي" استنادًا إلى موضع المتغير لاستدلال الأنواع. في المثال التالي، يعرف المصرّف أن e من النوع MouseEvent بسبب نوع الحدث click المعرّف في الملف lib.d.ts، الذي يحتوي على تصريحات محيطية لمختلف بُنى JavaScript الشائعة وDOM:

```
window.addEventListener('click', function (e) {}); // ستنتج للمتغير
```

توسيع النوع

توسيع الأنواع هو العملية التي تُسند فيها TypeScript نوعًا إلى متغير جرت تهيئته من دون تعليق توضيحي للنوع. ويسمح بالانتقال من الأنواع الضيقة إلى الأوسع، لا العكس. في المثال التالي:

```
let x = 'x'; // يستنتج TypeScript النوع string، وهو نوع واسع
let y: 'y' | 'x' = 'y'; // هو اتحاد أنواع حرفية y نوع
y = x; // غير صحيح: لا يمكن إسناد النوع 'string' إلى النوع '"x" | "y"'.
```

تُسند TypeScript النوع string إلى x استنادًا إلى القيمة الوحيدة المقدمة أثناء التهيئة (x)، وهذا مثال على التوسيع.

توفر TypeScript طرقًا للتحكم في عملية التوسيع، مثل استخدام "const".

Const

يؤدي استخدام الكلمة المفتاحية const عند التصريح عن متغير إلى استدلال نوع أضيق في TypeScript.

على سبيل المثال:

```
const x = 'x'; // يستنتج TypeScript أن نوع x هو 'x'، وهو نوع أضيق
let y: 'y' | 'x' = 'y';
y = x; // صحيح: النوع المستنتج للمتغير 'x' هو x
```

باستخدام const للتصريح عن المتغير x، يُضيّق نوعه إلى القيمة الحرفية المحددة 'x'. وبما أن نوع x قد ضُيّق، فيمكن إسناده إلى المتغير y من دون أي خطأ. والسبب في إمكان استدلال النوع هو أن متغيرات const لا يمكن إعادة الإسناد إليها، ولذلك يمكن تضييق نوعها إلى نوع حرفي محدد؛ وفي هذه الحالة، النوع الحرفي 'x'.

معدِّل Const على معاملات الأنواع

اعتبارًا من الإصدار 5.0 من TypeScript، يمكن تحديد السمة const على معامل نوع عام. ويسمح ذلك باستدلال أدق نوع ممكن. لنرَ مثالًا من دون استخدام const:

```
function identity<T>(value: T) {
    // لا يوجد const هنا
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // النوع المستنتج هو:
```

كما ترى، يُستدل على أن الخاصيتين a وb من النوع string.

والآن، لنرَ الفرق مع إصدار const:

```
function identity<const T>(value: T) {
    // استخدام المعدّل const مع معاملات النوع
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // النوع المستنتج هو:
```

يمكننا الآن ملاحظة أن الخاصيتين a وb تُستنتجان بوصفهما قيمتين نصيتين حرفيتين بدلًا من مجرد نوعي string.

توكيد Const

تتيح لك هذه الميزة إعلان متغير بنوع حرفي أكثر دقة استنادًا إلى قيمة تهيئته، ما يشير للمصرّف إلى أن القيمة ينبغي أن تُعامل بوصفها قيمة حرفية غير قابلة للتغيير. وفيما يلي بعض الأمثلة:

على خاصية واحدة:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

على كائن كامل:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

يمكن أن يكون هذا مفيدًا بصورة خاصة عند تعريف نوع tuple:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // صف readonly [1, 2, 3]
```

التعليق التوضيحي الصريح للنوع

يمكننا أن نكون محددين ونمرر نوعًا. في المثال التالي، الخاصية x من النوع number:

```
const v = {
    x: 1, // النوع المستنتج: number (توسيع)
```

```
};
v.x = 3; // صحيح
```

يمكننا جعل التعليق التوضيحي للنوع أكثر تحديدًا باستخدام اتحاد من الأنواع الحرفية:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // نوع x 3 | 2 | 1 :الآن اتحاد أنواع حرفية
};
v.x = 3; // صحيح
v.x = 100; // غير صحيح
```

تضييق النوع

تضييق الأنواع هو العملية التي يُضيّق فيها نوع عام إلى نوع أكثر تحديدًا في TypeScript. ويحدث ذلك عندما تحلل TypeScript الشيفرة وتحدد أن شروطًا أو عمليات معينة يمكنها تحسين معلومات النوع.

يمكن أن يحدث تضييق الأنواع بطرق مختلفة، منها:

الشروط

باستخدام العبارات الشرطية، مثل `if` أو `switch`، يمكن لـ TypeScript تضييق النوع استنادًا إلى نتيجة الشرط. على سبيل المثال:

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // النوع هو number، وقد ضُيّق بواسطة الشرط
}
```

طرح استثناء أو الإرجاع

يمكن استخدام طرح خطأ أو الإرجاع المبكر من فرع لمساعدة TypeScript على تضييق النوع. على سبيل المثال:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

تشمل الطرق الأخرى لتضييق الأنواع في TypeScript ما يلي:

- المعامل instanceof: يُستخدم للتحقق مما إذا كان كائن ما مثيلًا لصنف محدد.
- المعامل in: يُستخدم للتحقق مما إذا كانت خاصية موجودة في كائن.
- المعامل typeof: يُستخدم للتحقق من نوع قيمة في وقت التشغيل.
- الدوال المضمّنة مثل Array.isArray(): تُستخدم للتحقق مما إذا كانت القيمة مصفوفة.

الاتحاد المميَّز

يُعد استخدام "الاتحاد المميَّز" نمطًا في TypeScript تُضاف فيه "علامة" صريحة إلى الكائنات للتمييز بين الأنواع المختلفة داخل اتحاد. ويُشار إلى هذا النمط أيضًا باسم "الاتحاد الموسوم". في المثال التالي، تمثل الخاصية "type" هذه "العلامة":

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // النوع هو A
        case 'type_b':
            return input.value + 'extra'; // النوع هو B
    }
};
```

حراس الأنواع المعرَّفة من قِبل المستخدم

في الحالات التي لا تتمكن فيها TypeScript من تحديد نوع، يمكن كتابة دالة مساعدة تُعرف باسم "حارس نوع معرَّف من قِبل المستخدم". في المثال التالي، سنستخدم مُسنَد نوع لتضييق النوع بعد تطبيق عملية تصفية معينة:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // النوع هو (string | nul

const isValid = (item: string | null): item is string => item !=

const r2 = data.filter(isValid); // النوع صحيح الآن وهو string[]، بد
```

التضييق باستخدام Switch-true

تضيف TypeScript 5.3 التضييق باستخدام switch-true، ما يتيح لك استبدال سلاسل if/else الفوضوية بـ switch (true) باستخدام شروط منطقية. ويحسن ذلك قابلية القراءة مع استمرار تضييق الأنواع. وهو يشبه مطابقة الأنماط، لكنه أبسط.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

الأنواع الأولية

يدعم TypeScript سبعة أنواع أولية. يشير نوع البيانات الأولي إلى نوع ليس كائنًا ولا ترتبط به أي أساليب. في TypeScript، تكون جميع الأنواع الأولية غير قابلة للتغيير، ما يعني أنه لا يمكن تغيير قيمها بعد إسنادها.

string

يخزّن النوع البدائي `string` البيانات النصية، وتكون القيمة دائمًا محاطة بعلامتي اقتباس مزدوجتين أو مفردتين.

```
const x: string = 'x';
const y: string = 'y';
```

يمكن أن تمتد السلاسل النصية عبر عدة أسطر إذا أُحيطت بعلامة الفاصلة العليا المائلة (`):

```
let sentence: string = `xxx,
   yyy`;
```

boolean

يخزّن نوع البيانات `boolean` في TypeScript قيمة ثنائية، إما `true` وإما `false`.

```
const isReady: boolean = true;
```

number

يُمثَّل نوع البيانات `number` في TypeScript بقيمة فاصلة عائمة ذات 64 بت. ويمكن لنوع `number` تمثيل الأعداد الصحيحة والكسور. يدعم TypeScript أيضًا النظام السداسي عشري والثنائي والثماني، على سبيل المثال:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // 0 التمثيل الست عشري بالبادئة
const binary: number = 0b1010; // 0 يبدأ التمثيل الثنائي بالبادئة b
const octal: number = 0o633; // 0 يبدأ التمثيل الثماني بالبادئة o
```

bigint

يمثّل `bigint` قيمًا صحيحة يمكن أن تكون أكبر من الحد الأقصى للعدد الصحيح الآمن الذي يدعمه `number`، وهو 2^53 - 1.

يمكن إنشاء `bigint` باستدعاء الدالة المضمّنة `BigInt()` أو بإضافة `n` إلى نهاية أي قيمة عددية صحيحة حرفية:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

ملاحظات:

- لا يمكن مزج قيم `bigint` مع `number`، ولا يمكن استخدامها مع الكائن المضمّن `Math`؛ بل يجب تحويلها قسرًا إلى النوع نفسه.
- لا تتوفر قيم `bigint` إلا إذا كان إعداد الهدف ES2020 أو إصدارًا أحدث.

Symbol

الرموز هي معرّفات فريدة يمكن استخدامها مفاتيحَ للخصائص في الكائنات لمنع تعارض الأسماء.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null وundefined

يمثّل النوعان `null` و`undefined` كلاهما انعدام القيمة أو غياب أي قيمة.

يعني النوع `undefined` أن القيمة لم تُسنَد أو تُهيّأ، أو يشير إلى غياب غير مقصود للقيمة.

يعني النوع `null` أننا نعلم أن الحقل لا يحتوي على قيمة، ومن ثم تكون القيمة غير متاحة، وهو يشير إلى غياب مقصود للقيمة.

Array

المصفوفة `array` هي نوع بيانات يمكنه تخزين عدة قيم من النوع نفسه أو من أنواع مختلفة. ويمكن تعريفها باستخدام الصياغة الآتية:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // اتحاد
```

يدعم TypeScript المصفوفات المخصّصة للقراءة فقط باستخدام الصياغة الآتية:

```
const x: readonly string[] = ['a', 'b']; // المعدّل readonly
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // غير صحيح
```

يدعم TypeScript أنواع Tuple وأنواع Tuple المخصّصة للقراءة فقط:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

يمثّل نوع البيانات `any` حرفيًا «أي» قيمة، وهو النوع الافتراضي عندما يتعذر على TypeScript استدلال النوع أو عندما لا يكون محددًا.

عند استخدام any، يتخطى مصرّف TypeScript التحقق من الأنواع، ولذلك لا توجد سلامة للأنواع عند استخدام any. وبوجه عام، لا تستخدم any لإسكات المصرّف عند حدوث خطأ؛ بل ركّز بدلًا من ذلك على إصلاح الخطأ، لأن استخدام any يجعل من الممكن خرق العقود وفقدان فوائد الإكمال التلقائي في TypeScript.

قد يكون النوع any مفيدًا أثناء الانتقال التدريجي من JavaScript إلى TypeScript، إذ يمكنه إسكات المصرّف.

للمشروعات الجديدة، استخدم إعداد TypeScript الذي noImplicitAny، يمكّن TypeScript من إصدار أخطاء في المواضع التي يُستخدم فيها any أو يُستدل عليه.

عادةً ما يكون النوع any مصدرًا للأخطاء التي يمكنها إخفاء مشكلات حقيقية في أنواعك. تجنّب استخدامه قدر الإمكان.

التعليقات التوضيحية للأنواع

يمكن اختياريًا إضافة نوع إلى المتغيرات المصرّح بها باستخدام var وlet وconst:

```
const x: number = 1;
```

يجيد TypeScript استدلال الأنواع، ولا سيما الأنواع البسيطة، لذلك لا تكون هذه التصريحات ضرورية في معظم الحالات.

يمكن إضافة تعليقات توضيحية للأنواع إلى معاملات الدوال:

```
function sum(a: number, b: number) {
    return a + b;
}
```

فيما يأتي مثال يستخدم دالة مجهولة (تُسمّى أيضًا دالة لامبدا):

```
const sum = (a: number, b: number) => a + b;
```

يمكن الاستغناء عن هذه التعليقات التوضيحية عند وجود قيمة افتراضية للمعامل:

```
const sum = (a = 10, b: number) => a + b;
```

يمكن إضافة تعليقات توضيحية لنوع الإرجاع إلى الدوال:

```
const sum = (a = 10, b: number): number => a + b;
```

يفيد هذا خصوصًا مع الدوال الأكثر تعقيدًا، إذ يمكن أن تساعدك كتابة نوع الإرجاع قبل التنفيذ على التفكير مليًا في الدالة.

بوجه عام، فكّر في إضافة تعليقات توضيحية إلى تواقيع الأنواع، لا إلى المتغيرات المحلية داخل المتن، وأضف دائمًا أنواعًا إلى الكائنات الحرفية.

الخصائص الاختيارية

يمكن للكائن تحديد خصائص اختيارية بإضافة علامة استفهام ? إلى نهاية اسم الخاصية:

```
type X = {
    a: number;
    b?: number; // اختياري
};
```

يمكن تحديد قيمة افتراضية عندما تكون الخاصية اختيارية:

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

الخصائص المخصّصة للقراءة فقط

يمكن منع الكتابة إلى خاصية باستخدام المعدِّل readonly، الذي يضمن عدم إمكان إعادة كتابة الخاصية، لكنه لا يوفر أي ضمان بعدم قابلية التغيير الكاملة:

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

تواقيع الفهرسة

في TypeScript، يمكننا استخدام string وnumber وsymbol كتواقيع فهرسة:

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // النتيجة نفسها التي يعطيها k[1]
```

يرجى ملاحظة أن JavaScript يحوّل تلقائيًا فهرسًا من النوع number إلى فهرس من النوع string، ولذلك يُرجع k[1] أو k["1"] القيمة نفسها.

توسعة الأنواع

يمكن توسيع `interface` (بنسخ الأعضاء من نوع آخر):

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

يمكن أيضًا التوسيع من أنواع متعددة:

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

تعمل الكلمة المفتاحية `extends` فقط مع الواجهات والفئات، أما الأنواع فتستخدم التقاطع:

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

يمكن توسيع نوع باستخدام واجهة، ولكن ليس العكس:

```
type A = {
    a: string;
};
interface B extends A {
```

```
    b: string;
}
```

الأنواع الحرفية

النوع الحرفي هو مجموعة أحادية العنصر داخل نوع جماعي؛ فهو يعرّف قيمة دقيقة جدًا تكون قيمة بدائية في JavaScript.

الأنواع الحرفية في TypeScript هي الأعداد والسلاسل النصية والقيم المنطقية.

مثال على القيم الحرفية:

```
const a = 'a'; // نوع سلسلة نصية حرفية
const b = 1; // نوع قيمة عددية حرفية
const c = true; // نوع قيمة منطقية حرفية
```

تُستخدم الأنواع الحرفية النصية والعددية والمنطقية في الاتحادات وحراس الأنواع والأسماء المستعارة للأنواع. في المثال الآتي، يمكنك رؤية اسم مستعار لنوع اتحاد. يتكون 0 من القيم المحددة فقط، ولا تكون أي سلسلة نصية أخرى صالحة:

```
type 0 = 'a' | 'b' | 'c';
```

استدلال الأنواع الحرفية

استدلال الأنواع الحرفية ميزة في TypeScript تسمح باستدلال نوع متغير أو معامل استنادًا إلى قيمته.

في المثال الآتي، يمكننا أن نرى أن TypeScript يعدّ x نوعًا حرفيًا لأن القيمة لا يمكن تغييرها في أي وقت لاحق، بينما يُستنتج y بوصفه من النوع string لأنه يمكن تعديله في أي وقت لاحق.

```
const x = 'x'; // نوع حرفي 'x'، لأن هذه القيمة لا يمكن تغييرها
let y = 'y'; // النوع string، لأننا نستطيع تغيير هذه القيمة
```

في المثال الآتي، يمكننا أن نرى أن `o.x` استُنتج بوصفه `string` (لا قيمة حرفية من `a`) لأن TypeScript يعتبر أن القيمة يمكن تغييرها في أي وقت لاحق.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // أوسع هذا نوع string
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // لا يمكن إسناد وسيط من النوع 'string' من النوع
```

كما ترى، تطرح الشيفرة خطأ عند تمرير `o.x` إلى `fn` لأن X نوع أضيق.

يمكننا حل هذه المشكلة باستخدام توكيد نوع مع `const` أو النوع `X`:

```
let o = {
    x: 'a' as const,
};
```

أو:

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

`strictNullChecks` هو خيار لمصرّف TypeScript يفرض تحققًا صارمًا من القيم الخالية. عند تمكين هذا الخيار، لا يمكن إسناد `null` أو `undefined` إلى المتغيرات والمعاملات إلا إذا صُرّح بوضوح بأنها من ذلك النوع باستخدام نوع الاتحاد `null | undefined`. وإذا لم يُصرّح بوضوح بأن متغيرًا أو معاملًا يقبل قيمة خالية، فسيولّد TypeScript خطأ لمنع أخطاء وقت التشغيل المحتملة.

التعدادات

في TypeScript، يمثّل enum مجموعة من القيم الثابتة المسماة.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

يمكن تعريف التعدادات بطرائق مختلفة:

التعدادات العددية

في TypeScript، التعداد العددي هو تعداد تُسنَد فيه قيمة عددية إلى كل ثابت، بدءًا من 0 افتراضيًا.

```
enum Size {
    Small, // تبدأ القيمة من 0
    Medium,
    Large,
}
```

يمكن تحديد قيم مخصّصة بإسنادها صراحةً:

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

التعدادات النصية

في TypeScript، التعداد النصي هو تعداد تُسنَد فيه قيمة نصية إلى كل ثابت.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

ملاحظة: يسمح TypeScript باستخدام تعدادات غير متجانسة يمكن أن تتعايش فيها الأعضاء النصية والعددية.

التعدادات الثابتة

التعداد الثابت في TypeScript هو نوع خاص من التعدادات تكون فيه جميع القيم معروفة في وقت التصريف وتُضمَّن مباشرةً في كل موضع يُستخدم فيه التعداد، ما ينتج شيفرة أكثر كفاءة.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

سيُصرَّف إلى:

```
console.log('EN' /* Language.English */);
```

ملاحظات:

تحتوي التعدادات الثابتة على قيم مضمنة في الشيفرة، ما يؤدي إلى محو التعداد، ويمكن أن يكون ذلك أكثر كفاءة في المكتبات المستقلة، لكنه غير مرغوب فيه بوجه عام. كذلك لا يمكن أن تحتوي التعدادات الثابتة على أعضاء محسوبة.

الربط العكسي

يشير الربط العكسي في تعدادات TypeScript إلى القدرة على استرداد اسم عضو التعداد من قيمته. توجد لأعضاء التعداد افتراضيًا روابط أمامية من الاسم إلى القيمة، ولكن يمكن إنشاء روابط عكسية بتحديد قيم كل عضو

صراحةً. تفيد الروابط العكسية عندما تحتاج إلى البحث عن عضو تعداد باستخدام قيمته، أو عندما تحتاج إلى التكرار على جميع أعضاء التعداد. لاحظ أن أعضاء التعداد العددية وحدها هي التي تولّد روابط عكسية، بينما لا يُولَّد أي رابط عكسي لأعضاء التعداد النصية.

التعداد الآتي:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

يُصرَّف إلى:

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

لذلك، يعمل ربط القيم بالمفاتيح مع أعضاء التعداد العددية، ولكن ليس مع أعضاء التعداد النصية:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A
```

```
const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]); // هو نوع العنصر ضمنيًا 'any' من النوع
```

التعدادات المحيطية

التعداد المحيط في TypeScript هو نوع من التعدادات يُعرَّف في ملف تصريح (*.d.ts) من دون تنفيذ مرتبط به. ويتيح لك تعريف مجموعة من الثوابت المسماة التي يمكن استخدامها بطريقة آمنة من ناحية الأنواع عبر ملفات مختلفة من دون الحاجة إلى استيراد تفاصيل التنفيذ في كل ملف.

الأعضاء المحسوبة والثابتة

في TypeScript، العضو المحسوب هو عضو في تعداد تُحسب قيمته في وقت التشغيل، بينما العضو الثابت هو عضو تُحدَّد قيمته في وقت التصريف ولا يمكن تغييرها في أثناء وقت التشغيل. يُسمح بالأعضاء المحسوبة في التعدادات العادية، بينما يُسمح بالأعضاء الثابتة في كل من التعدادات العادية والثابتة.

```
// أعضاء ثابتة
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6، تُولّد أثناء الترجمة

// أعضاء محسوبة
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // عدد عشوائي يُولّد أثناء التشغيل
```

يُعبَّر عن التعدادات باتحادات تتألف من أنواع أعضائها. ويمكن تحديد قيم كل عضو من خلال تعبيرات ثابتة أو غير ثابتة، مع إسناد أنواع حرفية إلى الأعضاء التي تمتلك قيمًا ثابتة. للتوضيح، تأمل تصريح النوع E وأنواعه الفرعية E.A وE.B وE.C. في هذه الحالة، يمثّل E الاتحاد E.A | E.B | E.C.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // قيمة عددية حرفية
    B = 'bar', // سلسلة نصية حرفية
    C = identity(42), // قيمة محسوبة معتمة
}

console.log(E.C); //42
```

التضييق

تضييق الأنواع في TypeScript هو عملية تنقيح نوع متغير داخل كتلة شرطية. ويفيد ذلك عند التعامل مع أنواع الاتحاد، حيث يمكن أن يكون للمتغير أكثر من نوع واحد.

يتعرّف TypeScript على عدة طرائق لتضييق النوع:

حراس الأنواع باستخدام typeof

حارس النوع typeof هو حارس نوع محدد في TypeScript يتحقق من نوع متغير استنادًا إلى نوع JavaScript المضمّن الخاص به.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // نوع x هو number
    }
    return -1;
};
```

التضييق بحسب الصدقية

يعمل التضييق بحسب الصدقية في TypeScript من خلال التحقق مما إذا كان المتغير صادقًا أم زائفًا لتضييق نوعه وفقًا لذلك.

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

التضييق بحسب المساواة

يعمل التضييق بحسب المساواة في TypeScript من خلال التحقق مما إذا كان المتغير يساوي قيمة محددة أم لا، لتضييق نوعه وفقًا لذلك.

يُستخدم بالتزامن مع عبارات switch ومعاملات المساواة مثل === و==! و== و=! لتضييق الأنواع.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

التضييق باستخدام المعامل In

التضييق باستخدام المعامل in في TypeScript هو طريقة لتضييق نوع متغير استنادًا إلى وجود خاصية ضمن نوع المتغير.

```
type Dog = {
    name: string;
```

```
    breed: string;
};

type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

التضييق باستخدام instanceof

التضييق باستخدام المعامل `instanceof` في TypeScript هو طريقة لتضييق نوع متغير استنادًا إلى دالة المُنشئ الخاصة به، عن طريق التحقق مما إذا كان الكائن مثيلًا لفئة أو واجهة معينة.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
```

```
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

الإسنادات

تضييق الأنواع في TypeScript باستخدام الإسنادات هو طريقة لتضييق نوع متغير استنادًا إلى القيمة المسندة إليه. عندما تُسنَد قيمة إلى متغير، يستنتج TypeScript نوعه استنادًا إلى القيمة المسندة، ويضيّق نوع المتغير ليتطابق مع النوع المستنتج.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

تحليل تدفق التحكم

تحليل تدفق التحكم في TypeScript هو طريقة لتحليل تدفق الشيفرة تحليلاً ساكنًا لاستدلال أنواع المتغيرات، ما يسمح للمصرّف بتضييق أنواع تلك المتغيرات حسب الحاجة استنادًا إلى نتائج التحليل.

قبل TypeScript 4.4، كان تحليل تدفق الشيفرة يُطبَّق فقط على الشيفرة داخل عبارة if، ولكن بدءًا من TypeScript 4.4، يمكن تطبيقه أيضًا على التعبيرات الشرطية والوصول إلى الخصائص المميِّزة المشار إليها بصورة غير مباشرة من خلال متغيرات const.

على سبيل المثال:

```
const f1 = (x: unknown) => {
```

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
    obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar: numb
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

بعض الأمثلة التي لا يحدث فيها التضييق:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
        x.length; // خطأ، لا يحدث تضييق لأن isString ليس const
    }
};

const f6 = (
    obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar: numb
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
        obj.foo; // خطأ، لا يحدث تضييق لأن obj يُسنَد في جسم الدالة
    }
};
```

ملاحظات: يُحلّل ما يصل إلى خمسة مستويات من الإحالة غير المباشرة في التعبيرات الشرطية.

مسندات الأنواع

مسندات الأنواع في TypeScript هي دوال تُرجع قيمة منطقية وتُستخدم لتضييق نوع متغير إلى نوع أكثر تحديدًا.

```
const isString = (value: unknown): value is string => typeof val

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

يستنتج TypeScript 5.5 مسندات الأنواع تلقائيًا (مثل `x is T`) في دوال مثل `.filter`، ولذلك يعرف متى تُزال قيم مثل undefined، ما يوفر أنواعًا أدق وأخطاء أقل. ويعمل هذا مع عمليات التحقق الواضحة (مثل `x !== undefined`) ولكن ليس مع العمليات الملتبسة مثل `!!x`.

```
const nums = [1, null, 2].filter(x => x !== null);
```

الاتحادات المميَّزة

الاتحادات المميَّزة في TypeScript هي نوع من أنواع الاتحاد يستخدم خاصية مشتركة، تُعرف باسم المميِّز، لتضييق مجموعة الأنواع الممكنة للاتحاد.

```
type Square = {
    kind: 'square'; // مميّز
    size: number;
};
```

```ts
type Circle = {
    kind: 'circle'; // مميّز
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

النوع never

عندما يُضيَّق متغير إلى نوع لا يمكن أن يحتوي على أي قيم، يستنتج مصرّف TypeScript أن المتغير لا بد أن يكون من النوع never. ويرجع ذلك إلى أن النوع never يمثّل قيمة لا يمكن إنتاجها مطلقًا.

```ts
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
        // لا يمكن أن يكون سوى سلسلة نصية أو عدد never هنا هو نوع val
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
```

```
    }
};
```

التحقق من الشمول

التحقق من الشمول ميزة في TypeScript تضمن معالجة جميع الحالات الممكنة لاتحاد مميَّز في عبارة switch أو عبارة if.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
            console.log(exhaustiveCheck); // لن يُنفَّذ هذا السطر مطلقًا
    }
};
```

يُستخدم النوع never لضمان أن الحالة الافتراضية مستوفية لجميع الاحتمالات، وأن TypeScript سيُصدر خطأ إذا أُضيفت قيمة جديدة إلى النوع Direction من دون معالجتها في عبارة switch.

أنواع الكائنات

في TypeScript، تصف أنواع الكائنات شكل الكائن. فهي تحدد أسماء خصائص الكائن وأنواعها، وما إذا كانت تلك الخصائص مطلوبة أم اختيارية.

في TypeScript، يمكنك تعريف أنواع الكائنات بطريقتين أساسيتين:

تعرّف الواجهة شكل الكائن بتحديد أسماء خصائصه وأنواعها وما إذا كانت اختيارية.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

يعرّف الاسم المستعار للنوع، على غرار الواجهة، شكل الكائن. لكنه يستطيع أيضًا إنشاء نوع مخصّص جديد يستند إلى نوع موجود أو إلى مجموعة من الأنواع الموجودة. ويشمل ذلك تعريف أنواع الاتحاد وأنواع التقاطع وأنواع معقدة أخرى.

```
type Point = {
    x: number;
    y: number;
};
```

يمكن أيضًا تعريف نوع بصورة مجهولة:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

نوع Tuple (مجهول الاسم)

نوع Tuple هو نوع يمثّل مصفوفة ذات عدد ثابت من العناصر وأنواعها المقابلة. يفرض نوع Tuple عددًا محددًا من العناصر وأنواعًا محددة لكل منها بترتيب ثابت. تفيد أنواع Tuple عندما تريد تمثيل مجموعة من القيم ذات أنواع محددة، بحيث يكون لموضع كل عنصر في المصفوفة معنى محدد.

```
type Point = [number, number];
```

نوع Tuple المسمّى (الموسوم)

يمكن أن تتضمن أنواع Tuple تسميات أو أسماء اختيارية لكل عنصر. هذه التسميات مخصّصة لتحسين قابلية القراءة والمساعدة التي تقدمها الأدوات، ولا تؤثر في العمليات التي يمكنك إجراؤها عليها.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // صف مسمّى مع صف مجهول
```

Tuple ثابت الطول

Tuple ثابت الطول هو نوع محدد من Tuple يفرض عددًا ثابتًا من العناصر ذات أنواع محددة، ولا يسمح بأي تعديلات على طول Tuple بعد تعريفه.

تفيد أنواع Tuple ثابتة الطول عندما تحتاج إلى تمثيل مجموعة من القيم بعدد محدد من العناصر وأنواع محددة، وتريد ضمان عدم تغيير طول Tuple وأنواعه من دون قصد.

```
const x = [10, 'hello'] as const;
x.push(2); // خطأ
```

نوع الاتحاد

نوع الاتحاد هو نوع يمثّل قيمة يمكن أن تكون أحدَ عدة أنواع. ويُعبَّر عن أنواع الاتحاد باستخدام الرمز | بين كل نوع ممكن.

```
let x: string | number;
x = 'hello'; // صحيح
x = 123; // صحيح
```

أنواع التقاطع

نوع التقاطع هو نوع يمثّل قيمة تمتلك جميع خصائص نوعين أو أكثر. ويُعبَّر عن أنواع التقاطع باستخدام الرمز & بين كل نوع.

```ts
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // تقاطع

const j: J = {
    a: 'a',
    b: 'b',
};
```

فهرسة الأنواع

تشير فهرسة الأنواع إلى القدرة على تعريف أنواع يمكن فهرستها بمفتاح غير معروف مسبقًا، باستخدام توقيع فهرسة لتحديد نوع الخصائص غير المصرّح بها صراحةً.

```ts
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // يُرجع a
```

النوع من القيمة

يشير «النوع من القيمة» في TypeScript إلى الاستدلال التلقائي لنوع من قيمة أو تعبير من خلال استدلال النوع.

```ts
const x = 'x'; // يستنتج TypeScript أن 'x' نوع نصي حرفي مع const (
```

النوع من القيمة المرجعة للدالة

يشير «النوع من القيمة المرجعة للدالة» إلى القدرة على استدلال نوع الإرجاع لدالة تلقائيًا استنادًا إلى تنفيذها. ويتيح ذلك لـ TypeScript تحديد نوع القيمة التي تُرجعها الدالة من دون تعليقات توضيحية صريحة للنوع.

```
const add = (x: number, y: number) => x + y; // يستطيع TypeScrip
```

النوع من الوحدة

يشير «النوع من الوحدة» إلى القدرة على استخدام القيم التي تصدّرها الوحدة لاستدلال أنواعها تلقائيًا. عندما تصدّر وحدة قيمة من نوع محدد، يمكن لـ TypeScript استخدام تلك المعلومات لاستدلال نوع تلك القيمة تلقائيًا عند استيرادها إلى وحدة أخرى.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // نوع r هو number
```

الأنواع المعيّنة

تتيح لك الأنواع المعيّنة في TypeScript إنشاء أنواع جديدة استنادًا إلى نوع موجود، وذلك بتحويل كل خاصية باستخدام دالة تعيين. ومن خلال تعيين الأنواع الموجودة، يمكنك إنشاء أنواع جديدة تمثّل المعلومات نفسها بتنسيق مختلف. ولإنشاء نوع معيّن، تصل إلى خصائص نوع موجود باستخدام المعامل keyof ثم تعدّلها لإنتاج نوع جديد. في المثال الآتي:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
```

```
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

نعرّف MyMappedType لتعيين خصائص T، وإنشاء نوع جديد تكون فيه كل خاصية مصفوفةً من نوعها الأصلي. وباستخدام ذلك، ننشئ MyNewType لتمثيل المعلومات نفسها الموجودة في MyType، ولكن مع جعل كل خاصية مصفوفة.

معدِّلات الأنواع المعيّنة

تتيح معدِّلات الأنواع المعيّنة في TypeScript تحويل الخصائص داخل نوع موجود:

- `readonly` أو `+readonly`: يجعل الخاصية في النوع المعيّن مخصّصة للقراءة فقط.
- `-readonly`: يجعل الخاصية في النوع المعيّن قابلة للتغيير.
- `?`: يجعل الخاصية في النوع المعيّن اختيارية.

أمثلة:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // إءة فقط

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // ة للتعديل

type MyPartial<T> = { [P in keyof T]?: T[P] }; // ص محددة كاختيارية
```

الأنواع الشرطية

الأنواع الشرطية هي طريقة لإنشاء نوع يعتمد على شرط، حيث يُحدَّد النوع المراد إنشاؤه استنادًا إلى نتيجة الشرط. وتُعرَّف باستخدام الكلمة المفتاحية `extends` والمعامل الثلاثي للاختيار المشروط بين نوعين.

```ts
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // النوع true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // النوع false
```

الأنواع الشرطية التوزيعية

الأنواع الشرطية التوزيعية هي ميزة تسمح بتوزيع نوع على اتحاد من الأنواع من خلال تطبيق تحويل على كل عضو في الاتحاد على حدة. يمكن أن يكون هذا مفيدًا خصوصًا عند التعامل مع الأنواع المعيّنة أو الأنواع عالية الرتبة.

```ts
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number |
```

استدلال النوع باستخدام infer في الأنواع الشرطية

تُستخدم الكلمة المفتاحية `infer` في الأنواع الشرطية لاستدلال (استخراج) نوع معامل عام من نوع يعتمد عليه. ويتيح لك ذلك كتابة تعريفات أنواع أكثر مرونة وقابلية لإعادة الاستخدام.

```ts
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

الأنواع الشرطية المعرَّفة مسبقًا

في TypeScript، الأنواع الشرطية المعرَّفة مسبقًا هي أنواع شرطية مضمّنة توفرها اللغة. وقد صُممت لتنفيذ تحويلات شائعة للأنواع استنادًا إلى خصائص نوع معيّن.

`Exclude<UnionType, ExcludedType>`: يزيل هذا النوع من Type جميع الأنواع القابلة للإسناد إلى ExcludedType.

`Extract<Type, Union>`: يستخرج هذا النوع من Union جميع الأنواع القابلة للإسناد إلى Type.

`NonNullable<Type>`: يزيل هذا النوع null وundefined من Type.

`ReturnType<Type>`: يستخرج هذا النوع نوع الإرجاع لنوع دالة Type.

`Parameters<Type>`: يستخرج هذا النوع أنواع معاملات نوع دالة Type.

`Required<Type>`: يجعل هذا النوع جميع الخصائص في Type مطلوبة.

`Partial<Type>`: يجعل هذا النوع جميع الخصائص في Type اختيارية.

`Readonly<Type>`: يجعل هذا النوع جميع الخصائص في Type مخصّصة للقراءة فقط.

أنواع اتحادات القوالب

يمكن استخدام أنواع اتحادات القوالب لدمج النصوص ومعالجتها داخل نظام الأنواع، على سبيل المثال:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active"
```

النوع Any

النوع any هو نوع خاص (نوع فائق عام) يمكن استخدامه لتمثيل أي نوع من القيم (الأنواع الأولية، والكائنات، والمصفوفات، والدوال، والأخطاء، والرموز). وغالبًا ما يُستخدم في الحالات التي لا يكون فيها نوع القيمة معروفًا في وقت

التصريف، أو عند التعامل مع قيم من واجهات برمجة تطبيقات أو مكتبات خارجية لا تحتوي على تعريفات أنواع TypeScript.

باستخدام النوع any، فإنك تُعلم مصرّف TypeScript بأن القيم يجب أن تُمثّل من دون أي قيود. لتحقيق أقصى قدر من أمان الأنواع في شيفرتك، ضع ما يلي في الحسبان:

- احصر استخدام any في الحالات المحددة التي يكون فيها النوع مجهولًا بالفعل.
- لا تُعِد أنواع any من دالة، لأن ذلك يُضعف أمان الأنواع في الشيفرة التي تستخدمها.
- بدلًا من any، استخدم `@ts-ignore` إذا كنت بحاجة إلى إسكات المصرّف.

```
let value: any;
value = true; // صحيح
value = 7; // صحيح
```

النوع Unknown

في TypeScript، يمثّل النوع unknown قيمةً من نوع مجهول. وعلى خلاف النوع any الذي يسمح بأي نوع من القيم، يتطلب unknown فحصًا للنوع أو توكيدًا له قبل أن يمكن استخدامه بطريقة محددة، ولذلك لا يُسمح بإجراء أي عمليات على قيمة من النوع unknown من دون توكيد نوع أكثر تحديدًا أو تضييق النوع إليه أولًا.

لا يمكن إسناد النوع unknown إلا إلى any وإلى unknown نفسه، وهو بديل آمن من ناحية الأنواع للنوع any.

```
let value: unknown;

let value1: unknown = value; // صحيح
let value2: any = value; // صحيح
let value3: boolean = value; // غير صحيح
let value4: number = value; // غير صحيح
```

```
const add = (a: unknown, b: unknown): number | undefined =>
    typeof a === 'number' && typeof b === 'number' ? a + b : und
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

النوع Void

يُستخدم النوع void للإشارة إلى أن الدالة لا تُعيد قيمة.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

نوع Never

يمثّل النوع never القيم التي لا تحدث مطلقًا. ويُستخدم للدلالة على الدوال أو التعبيرات التي لا تعود مطلقًا أو ترمي خطأً.

على سبيل المثال، حلقة لا نهائية:

```
const infiniteLoop = (): never => {
    while (true) {
        // تنفيذ شيء ما
    }
};
```

رمي خطأ:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

النوع never مفيد في ضمان أمان الأنواع واكتشاف الأخطاء المحتملة في شيفرتك. فهو يساعد TypeScript على تحليل أنواع أكثر دقة واستدلالها عند استخدامه مع أنواع أخرى وتعليمات تدفق التحكم، على سبيل المثال:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // التحرك للأعلى
            break;
        case 'down':
            // التحرك للأسفل
            break;
        default:
            const exhaustiveCheck: never = direction;
            throw new Error(`Unhandled direction: ${exhaustiveCh
    }
};
```

الواجهة والنوع

الصياغة الشائعة

في TypeScript، تحدد الواجهات بنية الكائنات، مع تعيين أسماء الخصائص أو الأساليب التي يجب أن يحتوي عليها الكائن وأنواعها. والصياغة الشائعة لتعريف واجهة في TypeScript هي كما يلي:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

وبالمثل لتعريف نوع:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
```

```
    // ...
};
```

`interface InterfaceName` أو `type TypeName`: يحدد اسم الواجهة.
`property1: Type1`: يحدد خصائص الواجهة مع الأنواع المقابلة لها. يمكن تعريف خصائص متعددة، تفصل بين كل منها فاصلة منقوطة.
`method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: يحدد أساليب الواجهة. تُعرّف الأساليب بأسمائها، تليها قائمة معاملات بين قوسين ثم نوع الإرجاع. يمكن تعريف أساليب متعددة، تفصل بين كل منها فاصلة منقوطة.

مثال على واجهة:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

مثال على نوع:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

في TypeScript، تُستخدم الأنواع لتحديد شكل البيانات وفرض التحقق من الأنواع. توجد عدة صيغ شائعة لتعريف الأنواع في TypeScript، تبعًا لحالة الاستخدام المحددة. فيما يلي بعض الأمثلة:

الأنواع الأساسية

```
let myNumber: number = 123; // النوع number
let myBoolean: boolean = true; // النوع boolean
let myArray: string[] = ['a', 'b']; // مصفوفة سلاسل نصية
let myTuple: [string, number] = ['a', 123]; // صف
```

```
const x: { name: string; age: number } = { name: 'Simon', age: 7
```

أنواع الاتحاد والتقاطع

```
type MyType = string | number; // نوع اتحاد
let myUnion: MyType = 'hello'; // يمكن أن يكون سلسلة نصية
myUnion = 123; // أو عددًا

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // نوع تقاطع
let myCombined: CombinedType = { name: 'John', age: 25 }; // يتين
```

الأنواع الأولية المضمّنة

يحتوي TypeScript على عدة أنواع أولية مضمّنة يمكن استخدامها لتعريف المتغيرات ومعاملات الدوال وأنواع الإرجاع:

- `number`: يمثّل القيم العددية، بما في ذلك الأعداد الصحيحة وأعداد الفاصلة العائمة.
- `string`: يمثّل البيانات النصية.
- `boolean`: يمثّل القيم المنطقية التي يمكن أن تكون إما true أو false.
- `null`: يمثّل غياب قيمة.
- `undefined`: يمثّل قيمة لم تُسنَد أو لم تُعرّف.
- `symbol`: يمثّل معرّفًا فريدًا. تُستخدم الرموز عادةً بوصفها مفاتيحَ لخصائص الكائنات.
- `bigint`: يمثّل أعدادًا صحيحة بدقة اعتباطية.
- `any`: يمثّل نوعًا ديناميكيًا أو مجهولًا. يمكن للمتغيرات من النوع any الاحتفاظ بقيم من أي نوع، وهي تتجاوز التحقق من الأنواع.
- `void`: يمثّل غياب أي نوع. ويُستخدم عادةً نوعَ إرجاع للدوال التي لا تُعيد قيمة.

- `never`: يمثّل نوعًا للقيم التي لا تحدث مطلقًا. ويُستخدم عادةً نوعَ إرجاع للدوال التي ترمي خطأً أو تدخل في حلقة لا نهائية.

كائنات JS المضمّنة الشائعة

TypeScript هو مجموعة فائقة من JavaScript؛ ويشمل جميع كائنات JavaScript المضمّنة شائعة الاستخدام. يمكنك العثور على قائمة موسّعة بهذه الكائنات في موقع توثيق شبكة مطوري Mozilla (MDN): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

فيما يلي قائمة ببعض كائنات JavaScript المضمّنة شائعة الاستخدام:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set
- Promise
- Intl

التحميلات الزائدة

تتيح لك التحميلات الزائدة للدوال في TypeScript تعريف عدة تواقيع دوال لاسم دالة واحد، مما يتيح لك تعريف دوال يمكن استدعاؤها بطرائق متعددة. إليك مثالًا:

```
// تحميلات زائدة
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// التنفيذ
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // صحيح
sayHi(['aa', 'bb']); // صحيح
```

إليك مثالًا آخر على استخدام التحميلات الزائدة للدوال داخل class:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // تحميل زائد
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // التنفيذ
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
```

```
        } else if (Array.isArray(name)) {
            return name.map(name => `${this.message}, ${name}!`)
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

الدمج والتوسيع

يشير الدمج والتوسيع إلى مفهومين مختلفين يتعلقان بالتعامل مع الأنواع والواجهات.

يتيح لك الدمج ضم عدة تصريحات تحمل الاسم نفسه في تعريف واحد، على سبيل المثال، عندما تعرّف واجهة بالاسم نفسه عدة مرات:

```
interface X {
    a: string;
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
    b: 7,
};
```

يشير التوسيع إلى القدرة على توسيع الأنواع أو الواجهات الموجودة أو الوراثة منها لإنشاء أنواع أو واجهات جديدة. وهو آلية لإضافة خصائص أو أساليب إضافية إلى نوع موجود من دون تعديل تعريفه الأصلي. مثال:

```
interface Animal {
    name: string;
    eat(): void;
```

```
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

الاختلافات بين النوع والواجهة

دمج التصريحات (التوسعة):

تدعم الواجهات دمج التصريحات، ما يعني أنه يمكنك تعريف عدة واجهات بالاسم نفسه، وسيدمجها TypeScript في واجهة واحدة تحتوي على الخصائص والأساليب المجمّعة. وفي المقابل، لا تدعم الأنواع دمج التصريحات. يمكن أن يفيد ذلك عندما تريد إضافة وظائف إضافية أو تخصيص أنواع موجودة من دون تعديل التعريفات الأصلية أو ترقيع الأنواع المفقودة أو غير الصحيحة.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
```

```
    y: 'yy',
};
```

توسيع أنواع/واجهات أخرى:

يمكن لكل من الأنواع والواجهات توسيع أنواع/واجهات أخرى، لكن الصياغة مختلفة. مع الواجهات، تستخدم الكلمة المفتاحية extends لوراثة الخصائص والأساليب من واجهات أخرى. ومع ذلك، لا يمكن لواجهة توسيع نوع مركّب مثل نوع الاتحاد.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

بالنسبة إلى الأنواع، تستخدم المعامل & لدمج عدة أنواع في نوع واحد (تقاطع).

```
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
```

```
    y: 123,
    j: 'j',
};
```

أنواع الاتحاد والتقاطع:

تكون الأنواع أكثر مرونة عند تعريف أنواع الاتحاد والتقاطع. باستخدام الكلمة المفتاحية type، يمكنك بسهولة إنشاء أنواع اتحاد باستخدام المعامل | وأنواع تقاطع باستخدام المعامل &. وعلى الرغم من أن الواجهات يمكنها أيضًا تمثيل أنواع الاتحاد بصورة غير مباشرة، فإنها لا توفر دعمًا مضمّنًا لأنواع التقاطع.

```
type Department = 'dep-x' | 'dep-y'; // اتحاد

type Person = {
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // تقاطع
```

مثال باستخدام الواجهات:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // اتحاد واجهات
```

الصنف

الصياغة الشائعة للصنف

تُستخدم الكلمة المفتاحية `class` في TypeScript لتعريف صنف. يمكنك رؤية مثال أدناه:

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
            `Hello, my name is ${this.name} and I am ${this.age}
        );
    }
}
```

تُستخدم الكلمة المفتاحية `class` لتعريف صنف باسم "Person".

يحتوي الصنف على خاصيتين خاصتين: name من النوع `string` وage من النوع `number`.

يُعرّف المُنشئ باستخدام الكلمة المفتاحية `constructor`. ويأخذ name وage بوصفهما معاملين ويُسندهما إلى الخاصيتين المقابلتين.

يحتوي الصنف على أسلوب `public` باسم sayHi يسجّل رسالة ترحيب.

لإنشاء مثيل لصنف في TypeScript، يمكنك استخدام الكلمة المفتاحية `new` متبوعة باسم الصنف، ثم القوسين `()`. على سبيل المثال:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // الناتج: Hello, my name is John Doe and I am 2
```

المُنشئ

المُنشئات هي أساليب خاصة داخل الصنف تُستخدم لتهيئة خصائص الكائن عند إنشاء مثيل للصنف.

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age}
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

يمكن إجراء تحميل زائد لمُنشئ باستخدام الصياغة التالية:

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
```

```
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

في TypeScript، يمكن تعريف عدة تحميلات زائدة للمُنشئ، لكن لا يمكن أن يكون لديك سوى تنفيذ واحد يجب أن يكون متوافقًا مع جميع التحميلات الزائدة؛ ويمكن تحقيق ذلك باستخدام معامل اختياري.

```
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

المُنشئات الخاصة والمحمية

في TypeScript، يمكن وسم المُنشئات بأنها خاصة أو محمية، مما يقيّد إمكانية الوصول إليها واستخدامها.

المُنشئات الخاصة: لا يمكن استدعاؤها إلا من داخل الصنف نفسه. وغالبًا ما تُستخدم المُنشئات الخاصة في الحالات التي تريد فيها فرض نمط النسخة المفردة أو حصر إنشاء المثيلات في أسلوب مصنع داخل الصنف.

المُنشئات المحمية: تكون المُنشئات المحمية مفيدة عندما تريد إنشاء صنف أساسي ينبغي عدم إنشاء مثيل منه مباشرةً، لكن يمكن توسيعه بواسطة الأصناف الفرعية.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// ستؤدي محاولة إنشاء مثيل من الصنف الأساسي مباشرة إلى خطأ
// const baseObj = new BaseClass(); // خطأ: مُنشئ الصنف 'BaseClas

// إنشاء مثيل من الصنف المشتق
const derivedObj = new DerivedClass(10);
```

محددات الوصول

تُستخدم محددات الوصول private وprotected وpublic للتحكم في ظهور أعضاء الصنف وإمكانية الوصول إليها، مثل الخصائص والأساليب، في أصناف TypeScript. وتُعد هذه المحددات أساسية لفرض التغليف ووضع حدود للوصول إلى الحالة الداخلية للصنف وتعديلها.

يقيّد المحدد private الوصول إلى عضو الصنف بحيث لا يكون متاحًا إلا داخل الصنف الذي يحتويه.

يسمح المحدد protected بالوصول إلى عضو الصنف داخل الصنف الذي يحتويه وأصنافه المشتقة.

يوفر المحدد public وصولًا غير مقيّد إلى عضو الصنف، مما يسمح بالوصول إليه من أي مكان.

الجلب والضبط

دوال الجلب والضبط هي أساليب خاصة تتيح لك تعريف سلوك مخصص للوصول إلى خصائص الصنف وتعديلها. وهي تمكّنك من تغليف الحالة الداخلية للكائن وتوفير منطق إضافي عند جلب قيم الخصائص أو ضبطها. في TypeScript، تُعرّف دوال الجلب والضبط باستخدام الكلمتين المفتاحيتين get وset على التوالي. إليك مثالًا:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

الموصلات التلقائية في الأصناف

يضيف TypeScript الإصدار 4.9 دعمًا للموصلات التلقائية، وهي ميزة مرتقبة في ECMAScript. وهي تشبه خصائص الصنف، لكن يُصرّح عنها باستخدام الكلمة المفتاحية "accessor".

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

تُحوَّل الموصلات التلقائية إلى موصّلي get وset خاصين يعملان على خاصية لا يمكن الوصول إليها.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

في TypeScript، تشير الكلمة المفتاحية this إلى النسخة المثيلة الحالية من الصنف داخل أساليبه أو مُنشئاته. وهي تتيح لك الوصول إلى خصائص

الصنف وأساليبه وتعديلها من داخل نطاقه الخاص. وتوفر طريقة للوصول إلى الحالة الداخلية للكائن ومعالجتها داخل أساليبه الخاصة.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

خصائص المعاملات

تتيح لك خصائص المعاملات التصريح عن خصائص الصنف وتهيئتها مباشرةً ضمن معاملات المُنشئ، مما يجنبك الشيفرة النمطية المتكررة. على سبيل المثال:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
        // الكلمتان المفتاحيتان "private" و "public" في المُنشئ
        // تُصرّحان عن خصائص الصنف المقابلة وتُهيّئانها تلقائيًا.
    }
    public introduce(): void {
        console.log(
            `Hello, my name is ${this.name} and I am ${this.age}
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

الأصناف المجرّدة

تُستخدم الأصناف المجرّدة في TypeScript أساسًا للوراثة. وهي توفر طريقة لتعريف خصائص وأساليب مشتركة يمكن أن ترثها الأصناف الفرعية. يفيد ذلك عندما تريد تعريف سلوك مشترك وفرض تنفيذ أساليب معينة على الأصناف الفرعية. وهي توفر طريقة لإنشاء تسلسل هرمي من الأصناف، يوفّر فيه الصنف الأساسي المجرّد واجهة مشتركة ووظائف مشتركة للأصناف الفرعية.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // الناتج: Whiskers meows.
```

باستخدام الأنواع العامة

تتيح لك الأصناف ذات الأنواع العامة تعريف أصناف قابلة لإعادة الاستخدام يمكنها العمل مع أنواع مختلفة.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

المزخرفات

توفر المزخرفات آلية لإضافة بيانات وصفية أو تعديل السلوك أو التحقق أو توسيع وظائف العنصر المستهدف. وهي دوال تُنفّذ في وقت التشغيل. ويمكن تطبيق عدة مزخرفات على تصريح واحد.

المزخرفات ميزات تجريبية، والأمثلة التالية متوافقة فقط مع TypeScript الإصدار 5 أو أحدث عند استخدام ES6.

بالنسبة إلى إصدارات TypeScript السابقة للإصدار 5، يجب تمكينها باستخدام الخاصية `experimentalDecorators` في ملف `tsconfig.json` أو باستخدام `--experimentalDecorators` في سطر الأوامر (لكن المثال التالي لن يعمل).

تشمل بعض حالات الاستخدام الشائعة للمزخرفات ما يلي:

- مراقبة تغييرات الخصائص.
- مراقبة استدعاءات الأساليب.
- إضافة خصائص أو أساليب إضافية.
- التحقق في وقت التشغيل.
- التسلسل وإلغاء التسلسل تلقائيًا.
- التسجيل.
- التخويل والمصادقة.
- الحماية من الأخطاء.

ملاحظة: لا تسمح مزخرفات الإصدار 5 بزخرفة المعاملات.

أنواع المزخرفات:

مزخرفات الأصناف

تفيد مزخرفات الأصناف في توسيع صنف موجود، مثل إضافة خصائص أو أساليب، أو جمع مثيلات صنف. في المثال التالي، نضيف أسلوب `toString` يحوّل الصنف إلى تمثيل نصي.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}
```

```
@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* يطبع:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

مزخرف الخاصية

تفيد مزخرفات الخصائص في تعديل سلوك خاصية، مثل تغيير قيم التهيئة. في الشيفرة التالية، لدينا برنامج نصي يضبط خاصية بحيث تكون دائمًا بأحرف كبيرة:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
    prop1 = 'hello!';
```

```
}

console.log(new MyClass().prop1); // يطبع: HELLO!
```

مزخرف الأسلوب

تتيح لك مزخرفات الأساليب تغيير سلوك الأساليب أو تحسينه. فيما يلي مثال على مُسجِّل بسيط:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

    function replacementMethod(this: This, ...args: Args): Retu
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

new MyClass().sayHello();
```

يُسجّل ما يلي:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

مزخرفات دوال الجلب والضبط

تتيح لك مزخرفات دوال الجلب والضبط تغيير سلوك موصلات الصنف أو تحسينه. وهي مفيدة، على سبيل المثال، للتحقق من إسناد الخصائص. إليك مثالًا بسيطًا على مزخرف لدالة جلب:

```
function range<This, Return extends number>(min: number, max: nu
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}


class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
```

```
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // 10 :صحيح

const obj2 = new MyClass(999);
console.log(obj2.getValue); // يرمي: Invalid!
```

البيانات الوصفية للمزخرفات

تُبسّط البيانات الوصفية للمزخرفات عملية تطبيق البيانات الوصفية واستخدامها بواسطة المزخرفات في أي صنف. ويمكن للمزخرفات الوصول إلى خاصية جديدة للبيانات الوصفية في كائن السياق؛ ويمكن أن تُستخدم مفتاحًا لكل من القيم الأولية والكائنات. يمكن الوصول إلى معلومات البيانات الوصفية في الصنف عبر `Symbol.metadata`.

يمكن استخدام البيانات الوصفية لأغراض مختلفة، مثل تصحيح الأخطاء أو التسلسل أو حقن التبعيات باستخدام المزخرفات.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // بويض توافق بسيط

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
    | ClassMethodDecoratorContext; // سياق على بيانات الخاصية الوصفية

function setMetadata(_target: any, context: Context) {
    // ضبط كائن البيانات الوصفية بقيمة أولية
    context.metadata[context.name] = true;
}
```

```
class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // ل على البيانات الوصفية

console.log(JSON.stringify(metadata)); // {"bar":true,"baz":true
```

الوراثة

تشير الوراثة إلى الآلية التي يمكن لصنف من خلالها أن يرث خصائص وأساليب من صنف آخر يُعرف بالصنف الأساسي أو الصنف الفائق. ويمكن للصنف المشتق، الذي يُسمى أيضًا الصنف الابن أو الصنف الفرعي، توسيع وظائف الصنف الأساسي وتخصيصها بإضافة خصائص وأساليب جديدة أو تجاوز الخصائص والأساليب الموجودة.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}
```

```
class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// إنشاء مثيل من الصنف الأساسي
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// إنشاء مثيل من الصنف المشتق
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

لا يدعم TypeScript الوراثة المتعددة بالمعنى التقليدي، بل يسمح بالوراثة من صنف أساسي واحد. يدعم TypeScript واجهات متعددة. يمكن للواجهة تعريف عقد لبنية كائن، ويمكن للصنف تنفيذ عدة واجهات. وهذا يسمح للصنف بوراثة السلوك والبنية من مصادر متعددة.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
```

```
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

غالبًا ما يُشار إلى الكلمة المفتاحية `class` في TypeScript، كما في JavaScript، بوصفها سكرًا نحويًا. وقد قُدّمت في ECMAScript 2015 (ES6) لتوفير صياغة مألوفة أكثر لإنشاء الكائنات والتعامل معها بأسلوب قائم على الأصناف. ومع ذلك، من المهم ملاحظة أن TypeScript، بصفته مجموعة فائقة من JavaScript، يُصرَّف في النهاية إلى JavaScript الذي يظل قائمًا في جوهره على النماذج الأولية.

الأعضاء الساكنة

يحتوي TypeScript على أعضاء ساكنة. للوصول إلى الأعضاء الساكنة في صنف، يمكنك استخدام اسم الصنف متبوعًا بنقطة، من دون الحاجة إلى إنشاء كائن.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

تهيئة الخصائص

توجد عدة طرائق لتهيئة خصائص صنف في TypeScript:

مباشرةً:

في المثال التالي، ستُستخدم هذه القيم الأولية عند إنشاء مثيل للصنف.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

في المُنشئ:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

باستخدام معاملات المُنشئ:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
        // لا حاجة لإسناد القيم إلى الخصائص صراحةً.
    }
    log() {
        console.log(this.property2);
    }
}
```

```
const x = new MyClass();
x.log();
```

التحميل الزائد للأساليب

يتيح التحميل الزائد للأساليب للصنف أن يحتوي على عدة أساليب بالاسم نفسه، لكن بأنواع معاملات مختلفة أو بعدد مختلف من المعاملات. وهذا يتيح لنا استدعاء أسلوب بطرائق مختلفة بناءً على الوسائط المُمرَّرة.

```
class MyClass {
    add(a: number, b: number): number; // 1 توقيع التحميل الزائد
    add(a: string, b: string): string; // 2 توقيع التحميل الزائد

    add(a: number | string, b: number | string): number | string
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}

const r = new MyClass();
console.log(r.add(10, 5)); // 15 يطبع
```

الأنواع العامة

تتيح لك الأنواع العامة إنشاء مكوّنات ودوال قابلة لإعادة الاستخدام يمكنها العمل مع أنواع متعددة. وباستخدام الأنواع العامة، يمكنك تحديد معاملات للأنواع والدوال والواجهات، مما يتيح لها العمل على أنواع مختلفة من دون تحديدها صراحةً مسبقًا.

تتيح لك الأنواع العامة جعل الشيفرة أكثر مرونة وقابلية لإعادة الاستخدام.

النوع العام

لتعريف نوع عام، تستخدم قوسين زاويين (<>) لتحديد معاملات النوع، على سبيل المثال:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

الأصناف العامة

يمكن أيضًا تطبيق الأنواع العامة على الأصناف، وبهذه الطريقة يمكنها العمل مع أنواع متعددة باستخدام معاملات النوع. وهذا مفيد لإنشاء تعريفات أصناف قابلة لإعادة الاستخدام، يمكنها التعامل مع أنواع بيانات مختلفة مع الحفاظ على أمان الأنواع.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123
```

```
const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

قيود الأنواع العامة

يمكن تقييد المعاملات العامة باستخدام الكلمة المفتاحية extends متبوعةً بنوع أو واجهة يجب أن يستوفيها معامل النوع.

في المثال التالي، يجب أن تكون لدى T خاصية length ذات نوع صحيح حتى تكون صالحة:

```
const printLen = <T extends { length: number }>(value: T): void
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // غير صحيح
```

إحدى الميزات البارزة للأنواع العامة التي قُدّمت في الإصدار RC 3.4 هي استدلال أنواع الدوال ذات الرتبة الأعلى، الذي ينشر وسائط الأنواع العامة:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

تُسهّل هذه الوظيفة البرمجة بأسلوب pointfree الآمن من ناحية الأنواع، وهو أسلوب شائع في البرمجة الوظيفية.

التضييق السياقي للأنواع العامة

التضييق السياقي للأنواع العامة هو الآلية التي تتيح لمصرّف TypeScript تضييق نوع معامل عام استنادًا إلى السياق الذي يُستخدم فيه. ويكون مفيدًا عند العمل مع الأنواع العامة في العبارات الشرطية:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // تُضيَّق القيمة إلى النوع 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // تُضيَّق القيمة إلى النوع 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

الأنواع البنيوية الممحوة

في TypeScript، لا يلزم أن تتطابق الكائنات مع نوع محدد ودقيق. على سبيل المثال، إذا أنشأنا كائنًا يستوفي متطلبات واجهة، فيمكننا استخدام ذلك الكائن في المواضع التي تتطلب هذه الواجهة، حتى لو لم توجد أي صلة صريحة بينهما. مثال:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
```

```
    prop1: 'Origin',
};

log(obj); // صحيح
```

مساحات الأسماء

في TypeScript، تُستخدم مساحات الأسماء لتنظيم الشيفرة في حاويات منطقية، ومنع تعارض الأسماء، وتوفير طريقة لتجميع الشيفرة المترابطة معًا. يتيح استخدام الكلمة المفتاحية `export` الوصول إلى مساحة الاسم من خارج الوحدات.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

الرموز

الرموز هي نوع بيانات بدائي يمثل قيمة غير قابلة للتغيير ومضمونة التفرد عالميًا طوال مدة تشغيل البرنامج.

يمكن استخدام الرموز كمفاتيح لخصائص الكائنات، وهي توفّر طريقة لإنشاء خصائص غير قابلة للتعداد.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');
```

```
const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

أصبح استخدام الرموز كمفاتيح مسموحًا الآن في WeakMap وWeakSet.

توجيهات الشرطات المائلة الثلاث

توجيهات الشرطات المائلة الثلاث هي تعليقات خاصة تقدّم إلى المصرّف تعليمات حول كيفية معالجة ملف. تبدأ هذه التوجيهات بثلاث شرطات مائلة متتالية (///)، وتوضع عادةً في أعلى ملف TypeScript، ولا تؤثر في السلوك وقت التشغيل.

تُستخدم توجيهات الشرطات المائلة الثلاث للإشارة إلى التبعيات الخارجية، وتحديد سلوك تحميل الوحدات، وتمكين بعض ميزات المصرّف أو تعطيلها، وغير ذلك. فيما يلي بعض الأمثلة:

الإشارة إلى ملف تصريح:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

تحديد تنسيق الوحدة:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

تمكين خيارات المصرّف، وفي المثال التالي الوضع الصارم:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

معالجة الأنواع

إنشاء أنواع من أنواع أخرى

يمكن إنشاء أنواع جديدة من خلال تركيب الأنواع الموجودة أو معالجتها أو تحويلها.

أنواع التقاطع (&):

تتيح لك دمج أنواع متعددة في نوع واحد:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // تقاطع A و B
const obj: C = { foo: 42, bar: 'hello' };
```

أنواع الاتحاد (|):

تتيح لك تعريف نوع يمكن أن يكون واحدًا من عدة أنواع:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

الأنواع المعيّنة:

تتيح لك تحويل خصائص نوع موجود لإنشاء نوع جديد:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // صبح الخصائص للقراءة فقط
```

الأنواع الشرطية:

تتيح لك إنشاء أنواع استنادًا إلى بعض الشروط:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P : r
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

أنواع الوصول المفهرس

في TypeScript، يمكن الوصول إلى أنواع الخصائص الموجودة داخل نوع آخر ومعالجتها باستخدام فهرس، `Type[Key]`.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

أنواع الأدوات المساعدة

يمكن استخدام عدة أنواع أدوات مساعدة مضمّنة لمعالجة الأنواع، وفيما يلي قائمة بأكثرها استخدامًا:

<Awaited<T

ينشئ نوعًا يفك تغليف أنواع Promise بصورة متكررة.

```
type A = Awaited<Promise<string>>; // string
```

<Partial<T

ينشئ نوعًا تُعيَّن فيه جميع خصائص T على أنها اختيارية.

```ts
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?:
```

<Required<T

ينشئ نوعًا تُعيَّن فيه جميع خصائص T على أنها مطلوبة.

```ts
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

<Readonly<T

ينشئ نوعًا تُعيَّن فيه جميع خصائص T على أنها للقراءة فقط.

```ts
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // غير صحيح
```

<Record<K, T

ينشئ نوعًا يحتوي على مجموعة من الخصائص K من النوع T.

```ts
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

<Pick<T, K

ينشئ نوعًا بانتقاء الخصائص المحددة K من T.

```ts
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

<Omit<T, K

ينشئ نوعًا بحذف الخصائص المحددة K من T.

```ts
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

<Exclude<T, U

ينشئ نوعًا باستبعاد جميع قيم النوع U من T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

<Extract<T, U

ينشئ نوعًا باستخراج جميع قيم النوع U من T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

<NonNullable<T

ينشئ نوعًا باستبعاد null وundefined من T.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

<Parameters<T

يستخرج أنواع معاملات نوع الدالة T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

<ConstructorParameters<T

يستخرج أنواع معاملات نوع دالة المُنشئ T.

```
class Person {
    constructor(
        public name: string,
        public age: number
```

```
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof Pers
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

<ReturnType<T

يستخرج نوع الإرجاع لنوع الدالة T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

<InstanceType<T

يستخرج نوع المثيل لنوع الصنف T.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

<ThisParameterType<T>

يستخرج نوع معامل 'this' من نوع الدالة T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type PersonThisType = ThisParameterType<Person['greet']>; // Pe
```

<OmitThisParameter<T>

يزيل معامل 'this' من نوع الدالة T.

```
function capitalize(this: String) {
    return this[0].toUpperCase() + this.substring(1).toLowerCase
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; //
```

<ThisType<T>

يعمل كعلامة لنوع this سياقي.

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } & ThisType<Log
    hello: function () {
        this.log('some error'); // صحيح لأن "log" جزء من "this".
        this.update(); // غير صحيح
    },
};
```

<Uppercase<T

يحوّل اسم نوع الإدخال T إلى أحرف كبيرة.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

<Lowercase<T

يحوّل اسم نوع الإدخال T إلى أحرف صغيرة.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

<Capitalize<T

يحوّل الحرف الأول من اسم نوع الإدخال T إلى حرف كبير.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

<Uncapitalize<T

يحوّل الحرف الأول من اسم نوع الإدخال T إلى حرف صغير.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

<NoInfer<T

NoInfer هو نوع أداة مساعدة مصمم لمنع الاستدلال التلقائي للأنواع ضمن نطاق دالة عامة.

مثال:

```
// استنتاج الأنواع تلقائيًا ضمن نطاق دالة معمّمة.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
```

```
}
const r = fn(['a', 'b'], 'c'); // النوع هنا هو ("a" | "b" | "c")[]
```

باستخدام NoInfer:

```
// لمنع استنتاج النوع NoInfer دالة مثال تستخدم
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // خطأ: لا يمكن إسناد وسيط من النوع
```

مواضيع أخرى

الأخطاء ومعالجة الاستثناءات

تتيح لك TypeScript التقاط الأخطاء ومعالجتها باستخدام آليات JavaScript القياسية لمعالجة الأخطاء:

كتل Try-Catch-Finally:

```
try {
    // شيفرة قد ترمي خطأ
} catch (error) {
    // معالجة الخطأ
} finally {
    // و، شيفرة finally اختياري تُنفَّذ دائمًا
}
```

يمكنك أيضًا معالجة أنواع مختلفة من الأخطاء:

```
try {
    // شيفرة قد ترمي أنواعًا مختلفة من الأخطاء
} catch (error) {
    if (error instanceof TypeError) {
        // معالجة TypeError
```

```
    } else if (error instanceof RangeError) {
        // معالجة RangeError
    } else {
        // معالجة الأخطاء الأخرى
    }
}
```

أنواع الأخطاء المخصصة:

يمكن تحديد أخطاء أكثر تخصيصًا من خلال توسيع `class` Error:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

أصناف المزج

تتيح لك أصناف المزج دمج سلوك من أصناف متعددة وتركيبه في صنف واحد. وهي توفّر طريقة لإعادة استخدام الوظائف وتوسيعها دون الحاجة إلى سلاسل وراثة عميقة.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
```

```
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// توسيع MyClass لتشمل سلوك Identifiable و Selectable
interface MyClass extends Identifiable, Selectable {}

// دالة لتطبيق الخلطات على صنف
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {
        Object.getOwnPropertyNames(baseCtor.prototype).forEach(r
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                Object.defineProperty(source.prototype, name, de
            }
        });
    });
}

// تطبيق الخلطات على MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';

o.logId();
o.select();
```

ميزات اللغة غير المتزامنة

لأن TypeScript مجموعة شاملة من JavaScript، فهي تتضمن ميزات JavaScript المضمّنة للغة غير المتزامنة مثل:

الوعود:

الوعود هي طريقة لمعالجة العمليات غير المتزامنة ونتائجها باستخدام توابع مثل `then()` و`catch()` لمعالجة حالتي النجاح والخطأ.

لمعرفة المزيد: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

توفّر الكلمتان المفتاحيتان Async/await صياغةً تبدو أكثر تزامنًا عند العمل مع الوعود. تُستخدم الكلمة المفتاحية `async` لتعريف دالة غير متزامنة، وتُستخدم الكلمة المفتاحية `await` داخل دالة غير متزامنة لإيقاف التنفيذ مؤقتًا إلى أن تتم تسوية Promise بالقبول أو الرفض.

لمعرفة المزيد: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

واجهات API التالية مدعومة جيدًا في TypeScript:

واجهة Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

المكرِّرات والمولِّدات

تدعم TypeScript المكرِّرات والمولِّدات دعمًا جيدًا.

المكرِّرات هي كائنات تطبّق بروتوكول التكرار، وتوفّر طريقة للوصول إلى عناصر مجموعة أو تسلسل واحدًا تلو الآخر. وهي بنية تحتوي على مؤشر إلى العنصر التالي في عملية التكرار. ولديها تابع `next()` يعيد القيمة التالية في التسلسل إلى جانب قيمة منطقية تشير إلى ما إذا كان التسلسل قد أصبح done.

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}
```

```
const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

المولِّدات هي دوال خاصة تُعرّف باستخدام صياغة function* التي تبسّط إنشاء المكرِّرات. وهي تستخدم الكلمة المفتاحية yield لتعريف تسلسل القيم، وتوقف التنفيذ وتستأنفه تلقائيًا عند طلب القيم.

تسهّل المولِّدات إنشاء المكرِّرات، وهي مفيدة بوجه خاص عند العمل مع تسلسلات كبيرة أو غير منتهية.

مثال:

```
function* numberGenerator(start: number, end: number): Generator
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

تدعم TypeScript أيضًا المكرِّرات غير المتزامنة والمولِّدات غير المتزامنة.

لمعرفة المزيد:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

مرجع TsDocs وJSDoc

عند العمل مع قاعدة شيفرة JavaScript، يمكن مساعدة TypeScript على استدلال النوع الصحيح باستخدام تعليقات JSDoc مع شروح توضيحية إضافية لتوفير معلومات النوع.

مثال:

```
/**
 * تحسب قوة العدد المعطى
 * @constructor
 * @param {number} base - قيمة أساس التعبير
 * @param {number} exponent - قيمة أس التعبير
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number).
```

تتوفر الوثائق الكاملة على هذا الرابط:
https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

بدءًا من الإصدار 3.7، يمكن إنشاء تعريفات الأنواع .d.ts من صياغة JSDoc في JavaScript. يمكن العثور على مزيد من المعلومات هنا:
https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

types@

الحزم التابعة لمنظمة types@ هي اصطلاحات خاصة لتسمية الحزم، تُستخدم لتوفير تعريفات الأنواع لمكتبات JavaScript أو وحداتها الموجودة.

على سبيل المثال، سيؤدي استخدام:

```
npm install --save-dev @types/lodash
```

إلى تثبيت تعريفات الأنواع الخاصة بـ `lodash` في مشروعك الحالي.

للمساهمة في تعريفات الأنواع الخاصة بحزمة `@types`، يرجى إرسال طلب سحب إلى https://github.com/DefinitelyTyped/DefinitelyTyped.

JSX

JSX (JavaScript XML) هو امتداد لصياغة لغة JavaScript يتيح لك كتابة شيفرة شبيهة بـ HTML داخل ملفات JavaScript أو TypeScript. ويُستخدم عادةً في React لتعريف بنية HTML.

توسّع TypeScript إمكانات JSX من خلال توفير فحص الأنواع والتحليل الساكن.

لاستخدام JSX، تحتاج إلى تعيين خيار المصرّف `jsx` في ملف `tsconfig.json`. وفيما يلي خياران شائعان للإعداد:

- "preserve": يُنتج ملفات .jsx مع إبقاء JSX دون تغيير. يطلب هذا الخيار من TypeScript الحفاظ على صياغة JSX كما هي وعدم تحويلها أثناء عملية التصريف. يمكنك استخدام هذا الخيار إذا كانت لديك أداة منفصلة، مثل Babel، تتولى التحويل.
- "react": يمكّن تحويل JSX المضمّن في TypeScript. سيُستخدم React.createElement.

تتوفر جميع الخيارات هنا:
https://www.typescriptlang.org/tsconfig#jsx

وحدات ES6

تدعم TypeScript ES6 (ECMAScript 2015) والعديد من الإصدارات اللاحقة. ويعني ذلك أنه يمكنك استخدام صياغة ES6، مثل الدوال السهمية، وقوالب النصوص الحرفية، والأصناف، والوحدات، والتفكيك، وغير ذلك.

لتمكين ميزات ES6 في مشروعك، يمكنك تحديد الخاصية target في tsconfig.json.

مثال على إعداد:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

عامل الأُس في ES7

يحسب عامل الأُس (**) القيمة الناتجة من رفع المُعامل الأول إلى قوة المُعامل الثاني. ويعمل بصورة مشابهة لـ Math.pow()، لكن مع قدرة إضافية على قبول BigInts كمُعاملات. تدعم TypeScript هذا العامل بالكامل عند تعيين target في ملف tsconfig.json إلى es2016 أو إصدار أحدث.

```
console.log(2 ** (2 ** 2)); // 16
```

عبارة for-await-of

هذه ميزة JavaScript مدعومة بالكامل في TypeScript، وتتيح لك التكرار عبر الكائنات القابلة للتكرار غير المتزامنة مع الإصدار المستهدف es2018.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
```

```
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

الخاصية الوصفية new.target

يمكنك استخدام الخاصية الوصفية `new.target` في TypeScript، وهي تمكّنك من تحديد ما إذا استُدعيت دالة أو مُنشئ باستخدام العامل new. وتتيح لك اكتشاف ما إذا أُنشئ كائن نتيجة استدعاء مُنشئ.

```
class Parent {
    constructor() {
        console.log(new.target); // دالة المُنشئ المستخدمة لإنشاء مثيل
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

تعبيرات الاستيراد الديناميكي

يمكن تحميل الوحدات شرطيًا أو تحميلها تحميلًا كسولًا عند الطلب باستخدام مقترح ECMAScript للاستيراد الديناميكي، وهو مدعوم في TypeScript.

تكون صياغة تعبيرات الاستيراد الديناميكي في TypeScript كما يلي:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
        const widget = await import('./widget'); // ستيراد ديناميكي
        widget.render(container);
    }
}

renderWidget();
```

"tsc –watch"

يُشغّل هذا الأمر مصرّف TypeScript مع المعامل `--watch`، مع إمكانية إعادة تصريف ملفات TypeScript تلقائيًا كلما عُدّلت.

```
tsc --watch
```

بدءًا من TypeScript الإصدار 4.9، تعتمد مراقبة الملفات أساسًا على أحداث نظام الملفات، وتلجأ تلقائيًا إلى الاستقصاء إذا تعذر إنشاء مراقب قائم على الأحداث.

عامل التوكيد بعدم انعدام القيمة

عامل التوكيد بعدم انعدام القيمة (اللاحقة !)، الذي يُسمى أيضًا توكيدات الإسناد المحدد، هو ميزة في TypeScript تتيح لك تأكيد أن متغيرًا أو خاصية ليست null أو undefined، حتى إذا أشار تحليل TypeScript الساكن للأنواع إلى احتمال أن تكون كذلك. تتيح هذه الميزة إزالة أي فحص صريح.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
```

```
    console.log(`Name is ${person!.name}`);
};
```

التصريحات ذات القيم الافتراضية

تُستخدم التصريحات ذات القيم الافتراضية عندما يُسنَد إلى متغير أو معامل قيمة افتراضية. ويعني ذلك أنه إذا لم تُقدّم قيمة لذلك المتغير أو المعامل، فستُستخدم القيمة الافتراضية بدلًا منها.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

التسلسل الاختياري

يعمل عامل التسلسل الاختياري ?. مثل عامل النقطة العادي (.) للوصول إلى الخصائص أو التوابع. لكنه يتعامل بسلاسة مع قيم null أو undefined بإنهاء التعبير وإعادة undefined بدلًا من طرح خطأ.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

عامل الدمج المنعدم

يعيد عامل الدمج المنعدم ?? قيمة الطرف الأيمن إذا كان الطرف الأيسر null أو undefined؛ وإلا فيعيد قيمة الطرف الأيسر.

```
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

أنواع قوالب النصوص الحرفية

تتيح لك أنواع قوالب النصوص الحرفية معالجة قيم السلاسل النصية على مستوى النوع وإنشاء أنواع سلاسل نصية جديدة استنادًا إلى الأنواع الموجودة. وهي مفيدة لإنشاء أنواع أكثر تعبيرًا ودقة من العمليات القائمة على السلاسل النصية.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-englis
```

التحميل الزائد للدوال

يتيح لك التحميل الزائد للدوال تعريف عدة تواقيع للدالة نفسها، لكل منها أنواع معاملات وأنواع إرجاع مختلفة. عند استدعاء دالة محمّلة تحميلًا زائدًا، تستخدم TypeScript الوسائط المقدّمة لتحديد توقيع الدالة الصحيح:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
```

```
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

الأنواع العودية

النوع العودي هو نوع يمكنه الإشارة إلى نفسه. وهذا مفيد لتعريف بُنى بيانات لها بنية هرمية أو عودية (قد يكون تداخلها غير منتهٍ)، مثل القوائم المرتبطة، والأشجار، والرسوم البيانية.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

الأنواع الشرطية العودية

يمكن تعريف علاقات أنواع معقدة باستخدام المنطق والعودية في TypeScript. لنوضّح الأمر بعبارات بسيطة:

تتيح لك الأنواع الشرطية تعريف أنواع استنادًا إلى شروط منطقية:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a numbe
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

تعني العودية تعريف نوع يشير إلى نفسه داخل تعريفه:

```
type Json = string | number | boolean | null | Json[] | { [key:

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
```

```
        prop4: [],
    },
};
```

تجمع الأنواع الشرطية العودية بين المنطق الشرطي والعودية. وهذا يعني أن تعريف نوع يمكن أن يعتمد على نفسه من خلال منطق شرطي، ما ينشئ علاقات أنواع معقدة ومرنة.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 1
```

دعم وحدات ECMAScript في Node

أضاف Node.js دعم وحدات ECMAScript بدءًا من الإصدار 15.3.0، وتدعم TypeScript وحدات ECMAScript في Node.js منذ الإصدار 4.7. يمكن تمكين هذا الدعم باستخدام الخاصية `module` بالقيمة `nodenext` في ملف tsconfig.json. إليك مثالًا:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

يدعم Node.js امتدادين لملفات الوحدات: `mjs`. لوحدات ES و`cjs`. لوحدات CommonJS. والامتدادان المكافئان في TypeScript هما `mts`. لوحدات ES و`cts`. لوحدات CommonJS. عندما يحوّل مصرّف TypeScript هذه الملفات إلى JavaScript، فإنه ينشئ ملفات `mjs`. و`cjs`..

إذا كنت تريد استخدام وحدات ES في مشروعك، فيمكنك تعيين الخاصية type إلى "module" في ملف package.json. يطلب ذلك من Node.js التعامل مع المشروع على أنه مشروع وحدات ES.

بالإضافة إلى ذلك، تدعم TypeScript أيضًا تصريحات الأنواع في ملفات d.ts.. توفّر ملفات التصريحات هذه معلومات الأنواع للمكتبات أو الوحدات المكتوبة بلغة TypeScript، ما يتيح للمطورين الآخرين استخدامها مع ميزات فحص الأنواع والإكمال التلقائي في TypeScript.

دوال التوكيد

في TypeScript، دوال التوكيد هي دوال تشير إلى التحقق من شرط محدد استنادًا إلى قيمة الإرجاع الخاصة بها. في أبسط صورها، تفحص دالة التوكيد مُسندًا مقدّمًا وتطرح خطأ عندما تكون نتيجة المُسند false.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

أو يمكن التصريح عنها كتعبير دالة:

```
type AssertIsNumber = (value: unknown) => asserts value is numbe
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

تتشابه دوال التوكيد مع حراس الأنواع. كان الغرض من تقديم حراس الأنواع أصلًا إجراء عمليات تحقق وقت التشغيل وضمان نوع قيمة ضمن نطاق محدد. وبوجه خاص، حارس النوع هو دالة تقيّم مُسند نوع وتعيد قيمة منطقية تشير إلى ما إذا كان المُسند true أو false. وهذا يختلف قليلًا عن دوال التوكيد، إذ يكون القصد فيها طرح خطأ بدلًا من إعادة false عندما لا يتحقق المُسند.

مثال على حارس النوع:

```
const isNumber = (value: unknown): value is number => typeof val
```

أنواع الصفوف متغيرة الطول

أنواع الصفوف متغيرة الطول هي ميزة أُضيفت في الإصدار 4.0 من TypeScript، لذا لنبدأ بمراجعة مفهوم الصف:

نوع الصف هو مصفوفة ذات طول محدد، ويكون نوع كل عنصر فيها معروفًا:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

يعني مصطلح "متغير الطول" عدد معاملات غير محدد (قبول عدد متغير من الوسائط).

الصف متغير الطول هو نوع صف يمتلك جميع الخصائص السابقة، لكن شكله الدقيق لم يُحدَّد بعد:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

يمكننا أن نرى في الشيفرة السابقة أن شكل الصف تحدده T العامة التي جرى تمريرها.

يمكن للصفوف متغيرة الطول قبول عدة أنواع عامة، مما يجعلها شديدة المرونة:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, bool

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, bool
```

مع الصفوف الجديدة متغيرة الطول، يمكننا استخدام ما يلي:

- يمكن الآن أن تكون عوامل النشر في صياغة نوع الصف عامة، ولذلك يمكننا تمثيل العمليات ذات الرتبة الأعلى على الصفوف والمصفوفات حتى عندما لا نعرف الأنواع الفعلية التي نتعامل معها.
- يمكن أن تظهر عناصر الباقي في أي موضع داخل الصف.

مثال:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

الأنواع المُغلَّفة

تشير الأنواع المُغلَّفة إلى الكائنات المغلِّفة المستخدمة لتمثيل الأنواع الأولية في صورة كائنات. توفر هذه الكائنات المغلِّفة وظائف وأساليب إضافية غير متاحة مباشرةً على القيم الأولية.

عندما تصل إلى أسلوب مثل `charAt` أو `normalize` على قيمة أولية من النوع `string`، تغلفها JavaScript داخل كائن `String`، وتستدعي الأسلوب، ثم تتخلص من الكائن.

توضيح:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
```

```
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

تمثل TypeScript هذا الاختلاف بتوفير أنواع منفصلة للأنواع الأولية وأغلفة الكائنات المقابلة لها:

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

لا تكون الأنواع المُغلَّفة مطلوبة عادةً. تجنب استخدامها واستعمل الأنواع الأولية بدلًا منها، مثل `string` بدلًا من `String`.

التغاير والتغاير العكسي في TypeScript

يصف التغاير والتغاير العكسي العلاقات بين الأنواع داخل الأنواع العامة. النوع `Dog` هو `Animal`، لكن ليس كل `Animal` من نوع `Dog`.

التغاير (إنتاج القيم): يمكن استخدام دالة تُرجع `Dog` بدلًا من دالة تُرجع `Animal`. فكل كلب حيوان، لكن الدالة التي تُرجع أي حيوان قد تُرجع قطة.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;
```

```
produceAnimal = produceDog; // صحيح
produceDog = produceAnimal; // خطأ
```

التغاير العكسي (استقبال القيم): يمكن استخدام دالة تقبل أي `Animal` بدلًا من دالة تقبل `Dog`. فهي تقبل الكلاب أيضًا، أما الدالة المخصصة للكلاب فلا يمكنها التعامل بأمان مع كل الحيوانات.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // صحيح
consumeAnimal = consumeDog; // خطأ عند استخدام strictFunctionTypes
```

توضح الأسهم اتجاه الإسناد الآمن:

- Dog -> Animal
- (Producer<Dog> -> Producer<Animal> (covariance
- (Consumer<Animal> -> Consumer<Dog> (contravariance

المصفوفات القابلة للتعديل: يسمح TypeScript أيضًا بإسناد `Dog[]` إلى `Animal[]`. يشير المتغيران إلى المصفوفة نفسها؛ ويمكن إضافة `Animal` ليس كلبًا عبر النوع الأعم.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // مسموح، لكنه غير آمن
// يحتوي dogs الآن على كائن Animal ليس من النوع Dog.
```

خيار المترجم: مع تفعيل `strictFunctionTypes`، تُفحص معاملات الدوال المستقلة وفق التغاير العكسي. بدونه قد يقبل TypeScript الاتجاهين

(التغاير الثنائي) حتى إن لم يكونا آمنين. معاملات تعريفات الطرق والبناة استثناء من هذا الفحص.

تعليقات التغاير الاختيارية لمعاملات الأنواع

منذ TypeScript 4.7، تشير `out` إلى التغاير، و`in` إلى التغاير العكسي، و `in out` إلى الثبات. يستنتج TypeScript التغاير عادةً. يجب أن تتوافق التعليقات مع طريقة استخدام المعامل العام، ولا تغيّر سلوكه اعتباطيًا.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

تواقيع الفهرس ذات أنماط السلاسل القالبية

تتيح لنا تواقيع الفهرس ذات أنماط السلاسل القالبية تعريف تواقيع فهرس مرنة باستخدام أنماط السلاسل القالبية. تمكّننا هذه الميزة من إنشاء كائنات يمكن فهرستها باستخدام أنماط محددة من المفاتيح النصية، مما يوفر مزيدًا من التحكم والدقة عند الوصول إلى الخصائص ومعالجتها.

تسمح TypeScript، بدءًا من الإصدار 4.4، بتواقيع الفهرس للرموز وأنماط السلاسل القالبية.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
```

```
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // مفتاح رمز فريد
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

المعامل satisfies

يتيح لك المعامل satisfies التحقق مما إذا كان نوع معين يفي بواجهة أو شرط محدد. بعبارة أخرى، يضمن أن النوع يمتلك جميع الخصائص والأساليب المطلوبة لواجهة معينة. وهو طريقة للتأكد من أن متغيرًا يطابق تعريف نوع. إليك مثالًا:

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// توصيف النوع باستخدام `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// من استنتاج الأنواع بشكل صحيح في الأسطر التالية TypeScript لن يتمكن
user.attributes?.map(console.log); // موجودة في النوع 'map' الخاصية
user.nickName; // string | string[] | undefined


// توكيد النوع باستخدام `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// هنا أيضًا من استنتاج الأنواع بشكل صحيح TypeScript لن يتمكن
```

```
user2.attributes?.map(console.log); // الخاصية 'map' موجودة في النوع
user2.nickName; // string | string[] | undefined

// يمكننا الآن استنتاج الأنواع بشكل صحيح، باستخدام العامل `satisfies`
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log); // يستنتج TypeScript بشكل صحيح
user3.nickName; // يستنتج TypeScript بشكل صحيح: undefined
```

عمليات الاستيراد والتصدير الخاصة بالأنواع فقط

تتيح لك عمليات الاستيراد والتصدير الخاصة بالأنواع فقط استيراد الأنواع أو تصديرها دون استيراد أو تصدير القيم أو الدوال المرتبطة بتلك الأنواع. قد يفيد ذلك في تقليل حجم حزمتك.

لاستخدام عمليات الاستيراد الخاصة بالأنواع فقط، يمكنك استخدام الكلمة المفتاحية `import type`.

تسمح TypeScript باستخدام امتدادات ملفات التعريف والتنفيذ معًا (.ts و.mts و.cts و.tsx) في عمليات الاستيراد الخاصة بالأنواع فقط، بصرف النظر عن إعدادات `allowImportingTsExtensions`.

على سبيل المثال:

```
import type { House } from './house.ts';
```

الصيغ التالية مدعومة:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
```

```ts
export type { T };
export type { T } from './mod';
```

تصريح using والإدارة الصريحة للموارد

تصريح using هو ارتباط غير قابل للتغيير ضمن نطاق كتلة، ويشبه const، ويُستخدم لإدارة الموارد القابلة للتخلص منها. عند تهيئته بقيمة، يُسجَّل أسلوب Symbol.dispose الخاص بتلك القيمة ثم يُنفَّذ عند الخروج من نطاق الكتلة المحيطة.

يعتمد ذلك على ميزة إدارة الموارد في ECMAScript، وهي مفيدة لتنفيذ مهام التنظيف الأساسية بعد إنشاء الكائن، مثل إغلاق الاتصالات وحذف الملفات وتحرير الذاكرة.

ملاحظات:

- نظرًا إلى إضافتها حديثًا في الإصدار 5.2 من TypeScript، تفتقر معظم بيئات التشغيل إلى الدعم الأصلي. ستحتاج إلى إضافات توافق من أجل: Symbol.dispose وSymbol.asyncDispose وDisposableStack وAsyncDisposableStack وSuppressedError.
- بالإضافة إلى ذلك، ستحتاج إلى تهيئة tsconfig.json على النحو التالي:

```json
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

مثال:

```ts
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // تعويض توافق بسيط

const doWork = (): Disposable => {
    return {
```

```
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // يُصرَّح عن المورد
    console.log(2);
} // يُحرَّر المورد (مثلًا، يُقيَّم `work[Symbol.dispose]()`)

console.log(3);
```

ستطبع الشيفرة ما يلي:

```
1
2
disposed
3
```

يجب أن يلتزم المورد المؤهل للتخلص منه بواجهة Disposable:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

تسجل تصريحات using عمليات التخلص من الموارد في مكدس، مما يضمن التخلص منها بترتيب عكسي لترتيب التصريح:

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // يُحرَّر `C`، ثم `B`، ثم `A`.
```

يُضمن التخلص من الموارد حتى إذا وقع استثناء أو جرى تنفيذ شيفرة لاحقة. قد يؤدي هذا إلى احتمال أن تطرح عملية التخلص استثناءً قد يكبت استثناءً آخر. وللاحتفاظ بمعلومات عن الأخطاء المكبوتة، أُضيف استثناء أصلي جديد هو SuppressedError.

تصريح await using

يتعامل تصريح await using مع مورد قابل للتخلص منه بصورة غير متزامنة. يجب أن تحتوي القيمة على أسلوب Symbol.asyncDispose، وسُتنتظر نتيجته عند نهاية الكتلة.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // يُصرَّح عن المورد
} // يُحرَّر المورد (مثلًا، يُقيَّم `await work[Symbol.asyncDispose]()`)
```

لكي يكون المورد قابلًا للتخلص منه بصورة غير متزامنة، يجب أن يلتزم بواجهة Disposable أو AsyncDisposable:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose ??= Symbol('Symbol.asyncDispose'); // بسيط

class DatabaseConnection implements AsyncDisposable {
    // طريقة تُستدعى عند تحرير الكائن بشكل غير متزامن
    [Symbol.asyncDispose]() {
        // إغلاق الاتصال وإرجاع وعد
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
        await new Promise(resolve => setTimeout(resolve, 1000));
        console.log('Connection closed.');
```

```
        console.log('Connection closed.');
    }
}

async function doWork() {
    // إنشاء اتصال جديد وتحريره بشكل غير متزامن عند خروجه من النطاق
    await using connection = new DatabaseConnection(); // المورد
    console.log('Doing some work...');
} // يُحرَّر المورد (مثلًا، يُقيَّم `await connection[Symbol.asyncDispose]()

doWork();
```

تطبع الشيفرة ما يلي:

```
Doing some work...
Closing the connection...
Connection closed.
```

يُسمح باستخدام تصريحي `using` و`await using` في العبارات: `for` و`for-in` و`for-of` و`for-await-of` و`switch`.

سمات الاستيراد

تخبر سمات الاستيراد في TypeScript 5.3 (تسميات لعمليات الاستيراد) بيئة التشغيل بكيفية التعامل مع الوحدات (JSON وغيرها). يعزز ذلك الأمان من خلال ضمان وضوح عمليات الاستيراد، ويتوافق مع سياسة أمان المحتوى (CSP) من أجل تحميل الموارد بأمان أكبر. تضمن TypeScript صلاحيتها، لكنها تترك لبيئة التشغيل تفسيرها من أجل التعامل المحدد مع الوحدات.

مثال:

```
import config from './config.json' with { type: 'json' };
```

مع الاستيراد الديناميكي:

```
const config = import('./config.json', { with: { type: 'json' }
```

التحقق من صياغة التعبيرات النمطية

بدءًا من TypeScript 5.5.4، تتحقق TypeScript من القيم الحرفية للتعبيرات النمطية بحثًا عن الأخطاء الشائعة في وقت التصريف (مثل الصياغة غير الصالحة والمراجع الخلفية الخاطئة والميزات غير المدعومة في إصدار JavaScript المستهدف). يساعد ذلك في اكتشاف الأخطاء مبكرًا، لكنه لا يتحقق من سلاسل new RegExp("...").

```
let r = /(a)\2/; // خطأ: يشير هذا المرجع الخلفي إلى مجموعة غير موجودة.
```

import defer

يتيح لك `import defer` تحميل وحدة مع تأخير تنفيذها إلى أن تستخدم شيئًا منها فعليًا. يساعد ذلك على تجنب العمل والآثار الجانبية غير الضرورية.

- يعمل فقط مع: `import defer * as name from "module"`
- لا تُنفَّذ الشيفرة إلا عند الوصول إلى أحد التصديرات